PRACTICAL FILE

Advance Web Technologies

Student Name: Vansh Khandekar

Course: Master of Computer Applications (MCA) Semester - I

Janaprabha Institute of Engineering and Technology, Ramtek

Academic Year: 2025 - 2026

INDEX

Sr.No.	Practical Aim	Date	Sign.
1	To implement a professional landing page using React and Framer Motion....		
2	To design a secure relational schema in PostgreSQL (Supabase) and define TypeScript interf...		
3	To develop a state-driven resume builder with dynamic section management....		
4	To integrate Claude-3 Opus via OpenRouter API and implement system prompts for AI content ...		
5	To implement a rule-based engine for evaluating keyword density and ATS scores....		
6	To implement a browser-side PDF generation engine with high-fidelity layout rendering....		
7	To implement a custom React hook for debounced data persistence....		
8	To create an administrative control panel for monitoring platform-wide metrics....		
9	To implement secure JWT-based authentication using Supabase Auth....		
10	To design a scalable project architecture with global routing....		
11	To implement custom business logic hooks for resume management....		
12	To implement a custom floating AI assistant with persistent chat history....		
13	To implement responsive dashboard navigation....		
14	To implement automated resume score improvements UI....		

Practical No. 1

Aim: To implement a professional landing page using React and Framer Motion.

Source Code Implementation:

```
import { ArrowRight, BriefcaseBusiness, LayoutDashboard,
    Shield } from "lucide-react";
import { Card, CardContent, CardHeader, CardTitle } from
    "@/components/ui/card";
import { Dialog, DialogContent, DialogDescription,
    DialogHeader, DialogTitle, DialogTrigger } from
    "@/components/ui/dialog";

export function LandingHero() {
  const features = [
    { title: "AI Assistance", desc: "Get AI-powered
        suggestions to improve your resume.", icon: "■" },
    { title: "Professional Templates", desc: "Choose from
        20+ polished layouts.", icon: "■" },
    { title: "Easy Export", desc: "Download print-ready PDF
        in one click.", icon: "■" },
    { title: "ATS Optimized", desc: "Structure designed for
        ATS-friendly parsing.", icon: "■" },
    { title: "Fast Creation", desc: "Build a complete resume
        in under 10 minutes.", icon: "■" },
    { title: "For Students", desc: "Made for students,
        freshers, and internships.", icon: "■" },
  ];

  const team = [
    { initials: "VK", name: "Vansh Khandekar", role: "Team
        Leader" },
    { initials: "SC", name: "Shubham Chandekar", role:
        "Developer" },
    { initials: "RY", name: "Rahul Yenurkar", role:
        "Designer" },
    { initials: "PM", name: "Pranay Mende", role:
        "Researcher" },
  ];

  return (
    <div className="space-y-12">
      <div className="relative overflow-hidden rounded-2xl
          border border-primary/30 bg-gradient-to-r from-
          primary via-primary/90 to-secondary text-white
          shadow-lg shadow-primary/20">
        <div className="absolute inset-0 opacity-25">
          <div className="absolute -right-16 -top-20 h-56
```

```
      w-56 rounded-full bg-white/30 blur-3xl" />
  <div className="absolute -bottom-14 -left-14 h-44
      w-44 rounded-full bg-cyan-200/50 blur-3xl" />
</div>
<div className="relative p-6 md:p-8">
  <div className="flex flex-col lg:flex-row
      lg:items-center lg:justify-between gap-6">
    <div className="flex items-center gap-4">
      <div className="h-16 w-16 rounded-xl bg-
          white/20 backdrop-blur border border-
          white/40 flex items-center justify-
          center">
        <svg className="h-8 w-8 text-white"
            fill="none" stroke="currentColor"
            viewBox="0 0 24 24">
          <path strokeLinecap="round"
              strokeLinejoin="round" strokeWidth={2}
              d="M12 14l9-5-9-5-9 5 9 5z" />
        </svg>
      </div>
      <div>
        <h2 className="text-2xl md:text-3xl font-
            bold text-white">Janaprabha College,
            Ramtek</h2>
        <p className="text-lg text-white/90 font-
            medium">BCA 3rd Year • Academic Project
            2025-26</p>
      </div>
    </div>
    <div className="flex flex-wrap gap-3">
      {team.map((member) => (
        <div key={member.name} className="flex
            items-center gap-2 rounded-full border
            border-white/35 bg-white/20 px-4 py-2
            backdrop-blur">
          <div className="flex h-8 w-8 items-center
              justify-center rounded-full bg-white
              text-primary font-bold text-
              sm">{member.initials}</div>

          <div>
            <p className="text-sm font-semibold
                text-white">{member.name}</p>
            <p className="text-xs text-
                white/80">{member.role}</p>
          </div>
        </div>
      ))}
    </div>
```

```
    </div>
  </div>
</div>

<header className="relative overflow-hidden
    rounded-3xl border border-border/70 bg-card/85
    shadow-lg backdrop-blur">
  <div className="absolute inset-0 opacity-35">
    <div className="absolute -right-20 -top-24 h-80
        w-80 rounded-full bg-primary/20 blur-3xl" />
    <div className="absolute -bottom-16 -left-16 h-64
        w-64 rounded-full bg-secondary/20 blur-3xl" />
  </div>

  <div className="relative p-8 md:p-12 lg:p-16">
    <div className="max-w-3xl">
      <div className="flex items-center gap-2 mb-4">
        <span className="inline-flex items-center
            gap-1.5 rounded-full bg-primary px-4 py-2
            text-sm font-semibold text-primary-
            foreground shadow-sm">
          <svg className="h-4 w-4" fill="none"
              stroke="currentColor" viewBox="0 0 24
              24">
            <path strokeLinecap="round"
                strokeLinejoin="round" strokeWidth={2}
                d="M5 3v4M3 5h4M6
                17v4m-2-2h4m5-16l2.286 6.857L21
                12l-5.714 2.143L13 21l-2.286-6.857L5
                12l5.714-2.143L13 3z" />
          </svg>
          AI-Powered Resume Builder
        </span>
      </div>

      <h1 className="text-4xl font-bold tracking-tight
          md:text-5xl lg:text-6xl text-foreground">
        Build Your <span className="bg-gradient-to-r
            from-primary to-secondary bg-clip-text
            text-transparent">Professional
            Resume</span> in Minutes
      </h1>

      <p className="mt-4 max-w-2xl text-lg md:text-xl
          text-muted-foreground">
        Create stunning, ATS-friendly resumes with AI
            assistance. Stand out from the crowd and
            land your dream job.
```

```
</p>

<div className="mt-8 flex flex-col sm:flex-row
    gap-4">
  <a href="/create" className="inline-flex h-12
      items-center justify-center rounded-lg bg-
      primary px-8 font-semibold text-primary-
      foreground transition-colors hover:bg-
      primary/90">
    Start Building
    <ArrowRight className="ml-2 h-4 w-4" />
  </a>
  <a href="/dashboard" className="inline-flex
      h-12 items-center justify-center rounded-
      lg border border-border bg-background/80
      px-8 font-semibold text-foreground
      transition-colors hover:bg-muted/70">
    <LayoutDashboard className="mr-2 h-4 w-4" />
    Dashboard
  </a>

  <a href="/admin" className="inline-flex h-12
      items-center justify-center rounded-lg
      px-8 font-semibold text-primary
      hover:text-primary/80">
    <Shield className="mr-2 h-4 w-4" />
    Admin Panel
  </a>
  <Dialog>
    <DialogTrigger asChild>
      <button className="inline-flex h-12 items-
          center justify-center rounded-lg
          border border-border bg-background/80
          px-8 font-semibold text-foreground
          transition-colors hover:bg-muted/70">
        Project Details
      </button>
    </DialogTrigger>
    <DialogContent className="max-w-2xl">
      <DialogHeader>
        <DialogTitle>Project
            Details</DialogTitle>
        <DialogDescription>Academic evaluation
            snapshot</DialogDescription>
      </DialogHeader>
      <div className="grid gap-4 text-sm">
        <div><span className="font-
            semibold">Project Title:</span>
            ResumeGPT - Professional Resume
```

```
                    Builder</div>
                  <div><span className="font-
                      semibold">Course:</span> BCA 3rd
                      Year Major Project</div>
                  <div><span className="font-
                      semibold">Group Members:</span>
                      Vansh Khandekar, Shubham Chandekar,
                      Rahul Yenurkar, Pranay Mende</div>
                  <div><span className="font-
                      semibold">Problem Statement:</span>
                      Students struggle to create ATS-
                      friendly professional resumes
                      quickly.</div>
                  <div><span className="font-
                      semibold">Solution Overview:</span>
                      Guided form + AI writing assistant +
                      template system + PDF export +
                      resume scoring.</div>
                  <div><span className="font-semibold">Key
                      Features:</span> Live preview,
                      multiple templates, AI suggestions,
                      score analysis, admin
                      controls.</div>
                  <div><span className="font-
                      semibold">Future Scope:</span> Job-
                      role matching, keyword optimization,
                      interview preparation
                      assistant.</div>
                </div>
              </DialogContent>
            </Dialog>
          </div>
        </div>
      </div>
    </header>

    <section aria-label="Features">
      <h2 className="mb-6 bg-gradient-to-r from-primary
          to-secondary bg-clip-text text-2xl font-bold
          text-transparent">Why Choose ResumeGPT?</h2>
      <div className="grid gap-6 md:grid-cols-2 lg:grid-
          cols-3">
        {features.map((item) => (
          <div key={item.title} className="rounded-xl
              border border-border bg-card/80 p-6
              transition-colors hover:border-primary/60">
            <div className="mb-4 flex h-12 w-12 items-
                center justify-center rounded-xl bg-
```

```
            primary/10 text-2xl">
          {item.icon}
        </div>
        <h3 className="font-bold text-lg text-
            foreground">{item.title}</h3>
        <p className="text-muted-foreground
            mt-2">{item.desc}</p>
      </div>
    ))}
  </div>
</section>

<section aria-label="Stats" className="grid gap-6
    sm:grid-cols-3">
  {[
    { label: "Templates Available", value: "20+" },
    { label: "AI-Powered", value: "Yes" },
    { label: "PDF Export", value: "✓" }
  ].map((stat, i) => (
    <div key={i} className="rounded-xl border border-
        border bg-card/80 p-6 text-center">
      <p className="bg-gradient-to-r from-primary to-
          secondary bg-clip-text text-4xl font-
          extrabold text-transparent">{stat.value}</p>
      <p className="text-foreground mt-2 font-
          semibold">{stat.label}</p>
    </div>
  ))}
</section>

<section aria-label="Templates Preview">
  <h2 className="mb-6 bg-gradient-to-r from-primary
      to-secondary bg-clip-text text-2xl font-bold
      text-transparent">Templates Preview</h2>
  <div className="grid gap-4 md:grid-cols-5">
    {["Clean", "Professional", "ATS-friendly",
        "Modern", "Two-column"].map((name) => (
      <Card key={name} className="bg-card/80">
        <CardHeader className="pb-2">
          <CardTitle className="text-
              sm">{name}</CardTitle>
        </CardHeader>
        <CardContent>
          <div className="h-20 rounded-md border bg-
              muted/40" />
        </CardContent>
      </Card>
    ))}
  </div>
```

```
</section>

<section aria-label="AI and Score Preview"
    className="grid gap-4 md:grid-cols-2">
  <Card className="bg-card/80">
    <CardHeader>
      <CardTitle className="text-base">AI Assistant
          Preview</CardTitle>
    </CardHeader>
    <CardContent className="text-sm text-muted-
        foreground">
      Floating assistant helps generate summary,
          improve experience lines, suggest skills,
          and give resume tips.
    </CardContent>
  </Card>
  <Card className="bg-card/80">
    <CardHeader>
      <CardTitle className="text-base">Resume Score
          Preview</CardTitle>
    </CardHeader>
    <CardContent className="text-sm text-muted-
        foreground">
      AI-powered score out of 100 with section
          analysis and actionable improvement
          suggestions.

    </CardContent>
  </Card>
</section>

<section aria-label="Quick Links">
  <h2 className="mb-6 bg-gradient-to-r from-primary
      to-secondary bg-clip-text text-2xl font-bold
      text-transparent">Quick Links</h2>
  <div className="grid gap-4 sm:grid-cols-2">
    <a href="/admin" className="block">
      <div className="flex cursor-pointer items-center
          justify-between rounded-xl border border-
          border bg-card/80 p-5 transition-colors
          hover:border-primary/60">
        <div className="flex items-center gap-4">
          <div className="flex h-14 w-14 items-center
              justify-center rounded-xl bg-primary
              text-primary-foreground">
            <Shield className="h-7 w-7" />
          </div>
          <div>
            <p className="font-bold text-xl text-
```

```
                    foreground">Admin Panel</p>
                  <p className="text-muted-
                    foreground">Manage settings</p>
                </div>
              </div>
              <ArrowRight className="h-5 w-5 text-primary"
                />
            </div>
          </a>
          <a href="/dashboard" className="block">
            <div className="flex cursor-pointer items-center
                justify-between rounded-xl border border-
                border bg-card/80 p-5 transition-colors
                hover:border-primary/60">
              <div className="flex items-center gap-4">
                <div className="flex h-14 w-14 items-center
                    justify-center rounded-xl bg-secondary
                    text-secondary-foreground">
                  <BriefcaseBusiness className="h-7 w-7" />
                </div>
                <div>
                  <p className="font-bold text-xl text-
                    foreground">Dashboard</p>
                  <p className="text-muted-foreground">View
                    resumes</p>
                </div>
              </div>
              <ArrowRight className="h-5 w-5 text-primary"
                />
            </div>
          </a>
        </div>
      </section>
    </div>
  );
}
```

Result Output:

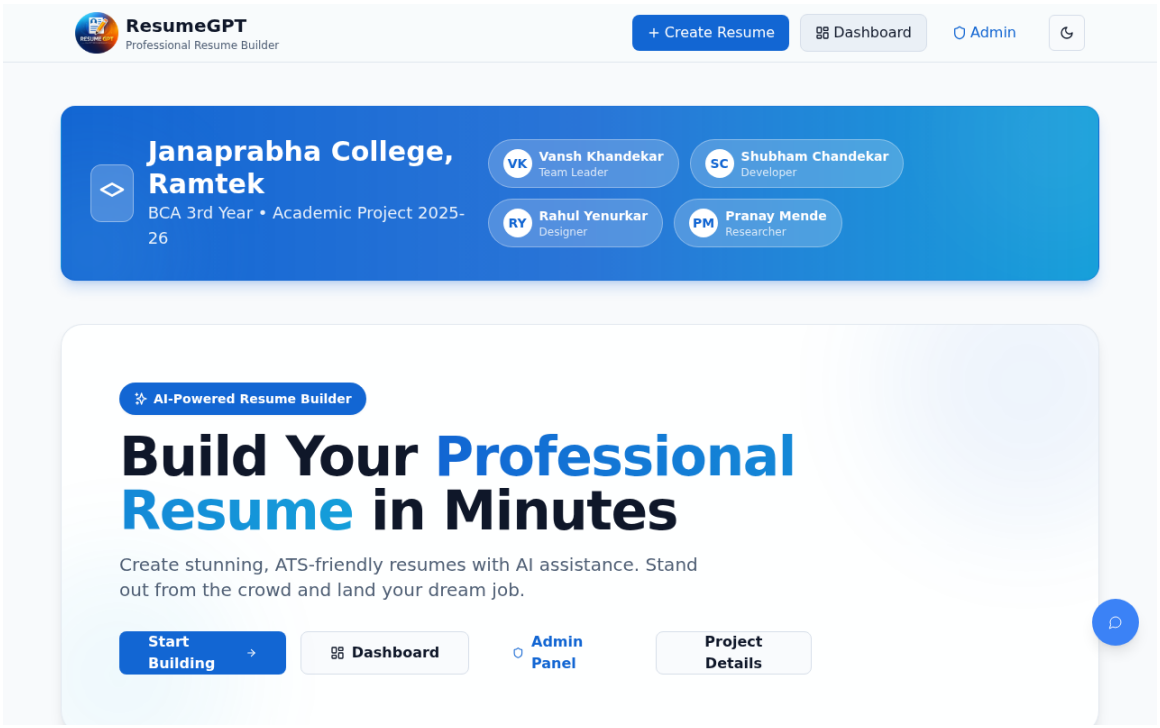

Figure 1: Practical System Output

Practical No. 2

Aim: To design a secure relational schema in PostgreSQL (Supabase) and define TypeScript interfaces.

Source Code Implementation:

```
export type Json =
  | string
  | number
  | boolean
  | null
  | { [key: string]: Json | undefined }
  | Json[]

export type Database = {
  // Allows to automatically instantiate createClient with
      right options
  // instead of createClient<Database, { PostgrestVersion:
      'XX' }>(URL, KEY)
  __InternalSupabase: {
    PostgrestVersion: "14.1"
  }
  public: {
    Tables: {
      admin_settings: {
        Row: {
          ai_enabled: boolean
          ai_model: string
          ai_provider: string
          created_at: string
          id: number
          max_lines: number
          updated_at: string
        }
        Insert: {
          ai_enabled?: boolean
          ai_model?: string
          ai_provider?: string
          created_at?: string
          id?: number
          max_lines?: number
          updated_at?: string
        }
        Update: {
          ai_enabled?: boolean
          ai_model?: string
          ai_provider?: string
```

```
      created_at?: string
      id?: number
      max_lines?: number
      updated_at?: string
    }
    Relationships: []

  }
  gemini_api_keys: {
    Row: {
      api_key: string
      created_at: string
      id: string
      is_active: boolean
      label: string | null
      priority: number
      updated_at: string
    }
    Insert: {
      api_key: string
      created_at?: string
      id?: string
      is_active?: boolean
      label?: string | null
      priority?: number
      updated_at?: string
    }
    Update: {
      api_key?: string
      created_at?: string
      id?: string
      is_active?: boolean
      label?: string | null
      priority?: number
      updated_at?: string
    }
    Relationships: []
  }
  user_roles: {
    Row: {
      created_at: string
      id: string
      role: Database["public"]["Enums"]["app_role"]
      user_id: string
    }
    Insert: {
      created_at?: string
      id?: string
      role: Database["public"]["Enums"]["app_role"]
```

```
      user_id: string
    }
    Update: {

      created_at?: string
      id?: string
      role?: Database["public"]["Enums"]["app_role"]
      user_id?: string
    }
    Relationships: []
  }
  profiles: {
    Row: {
      id: string
      full_name: string | null
      avatar_url: string | null
      plan: 'free' | 'pro' | 'enterprise'
      ai_calls_used: number
      resumes_created: number
      created_at: string
      updated_at: string
    }
    Insert: {
      id: string
      full_name?: string | null
      avatar_url?: string | null
      plan?: 'free' | 'pro' | 'enterprise'
      ai_calls_used?: number
      resumes_created?: number
      created_at?: string
      updated_at?: string
    }
    Update: {
      id?: string
      full_name?: string | null
      avatar_url?: string | null
      plan?: 'free' | 'pro' | 'enterprise'
      ai_calls_used?: number
      resumes_created?: number
      created_at?: string
      updated_at?: string
    }
    Relationships: []
  }
  resumes: {
    Row: {
      id: string
      user_id: string
      title: string
```

```
      data: any
      template_id: string
      section_order: string[]
      section_enabled: Record<string, boolean>
      last_score: number | null
      is_archived: boolean
      created_at: string
      updated_at: string
    }
    Insert: {
      id?: string
      user_id: string
      title?: string
      data?: any
      template_id?: string
      section_order?: string[]
      section_enabled?: Record<string, boolean>
      last_score?: number | null
      is_archived?: boolean
      created_at?: string
      updated_at?: string
    }
    Update: {
      id?: string
      user_id?: string
      title?: string
      data?: any
      template_id?: string
      section_order?: string[]
      section_enabled?: Record<string, boolean>
      last_score?: number | null
      is_archived?: boolean
      created_at?: string
      updated_at?: string
    }
    Relationships: []
  }
  usage_logs: {
    Row: {
      id: string
      user_id: string
      action: string
      metadata: any | null
      created_at: string
    }

    Insert: {
      id?: string
      user_id: string
```

```
          action: string
          metadata?: any | null
          created_at?: string
        }
        Update: {
          id?: string
          user_id?: string
          action?: string
          metadata?: any | null
          created_at?: string
        }
        Relationships: []
      }
    }
    Views: {
      [_ in never]: never
    }
    Functions: {
      has_role: {
        Args: {
          _role: Database["public"]["Enums"]["app_role"]
          _user_id: string
        }
        Returns: boolean
      }
      is_admin: { Args: never; Returns: boolean }
    }
    Enums: {
      app_role: "admin" | "user"
    }
    CompositeTypes: {
      [_ in never]: never
    }
  }
}

type DatabaseWithoutInternals = Omit<Database,
    "__InternalSupabase">

type DefaultSchema = DatabaseWithoutInternals[Extract<keyof
    Database, "public">]

export type Tables<
  DefaultSchemaTableNameOrOptions extends

    | keyof (DefaultSchema["Tables"] &
        DefaultSchema["Views"])
    | { schema: keyof DatabaseWithoutInternals },
  TableName extends DefaultSchemaTableNameOrOptions extends
```

```
      {
    schema: keyof DatabaseWithoutInternals
  }
    ? keyof (DatabaseWithoutInternals[DefaultSchemaTableName
        OrOptions["schema"]]["Tables"] &
        DatabaseWithoutInternals[DefaultSchemaTableNameOrOpt
            ions["schema"]]["Views"])
    : never = never,
> = DefaultSchemaTableNameOrOptions extends {
  schema: keyof DatabaseWithoutInternals
}
  ? (DatabaseWithoutInternals[DefaultSchemaTableNameOrOption
      s["schema"]]["Tables"] &
      DatabaseWithoutInternals[DefaultSchemaTableNameOrOptio
          ns["schema"]]["Views"])[TableName] extends {
      Row: infer R
    }
    ? R
    : never
  : DefaultSchemaTableNameOrOptions extends keyof
      (DefaultSchema["Tables"] &
        DefaultSchema["Views"])
    ? (DefaultSchema["Tables"] &
DefaultSchema["Views"])[DefaultSchemaTableNameOrOptions]
            extends {
        Row: infer R
      }
      ? R
      : never
    : never

export type TablesInsert<
  DefaultSchemaTableNameOrOptions extends
    | keyof DefaultSchema["Tables"]
    | { schema: keyof DatabaseWithoutInternals },
  TableName extends DefaultSchemaTableNameOrOptions extends
      {
    schema: keyof DatabaseWithoutInternals
  }
    ? keyof DatabaseWithoutInternals[DefaultSchemaTableNameO
        rOptions["schema"]]["Tables"]
    : never = never,
> = DefaultSchemaTableNameOrOptions extends {
  schema: keyof DatabaseWithoutInternals
}
  ? DatabaseWithoutInternals[DefaultSchemaTableNameOrOptions
      ["schema"]]["Tables"][TableName] extends {
      Insert: infer I
```

```
    }
    ? I
    : never
  : DefaultSchemaTableNameOrOptions extends keyof
      DefaultSchema["Tables"]

    ?
        DefaultSchema["Tables"][DefaultSchemaTableNameOrOpti
        ons] extends {
        Insert: infer I
      }
      ? I
      : never
    : never

export type TablesUpdate<
  DefaultSchemaTableNameOrOptions extends
    | keyof DefaultSchema["Tables"]
    | { schema: keyof DatabaseWithoutInternals },
  TableName extends DefaultSchemaTableNameOrOptions extends
      {
    schema: keyof DatabaseWithoutInternals
  }
    ? keyof DatabaseWithoutInternals[DefaultSchemaTableNameO
        rOptions["schema"]]["Tables"]
    : never = never,
> = DefaultSchemaTableNameOrOptions extends {
  schema: keyof DatabaseWithoutInternals
}
  ? DatabaseWithoutInternals[DefaultSchemaTableNameOrOptions
      ["schema"]]["Tables"][TableName] extends {
      Update: infer U
    }
    ? U
    : never
  : DefaultSchemaTableNameOrOptions extends keyof
      DefaultSchema["Tables"]
    ?
        DefaultSchema["Tables"][DefaultSchemaTableNameOrOpti
        ons] extends {
        Update: infer U
      }
      ? U
      : never
    : never

export type Enums<
  DefaultSchemaEnumNameOrOptions extends
    | keyof DefaultSchema["Enums"]
```

```
    | { schema: keyof DatabaseWithoutInternals },
  EnumName extends DefaultSchemaEnumNameOrOptions extends {
    schema: keyof DatabaseWithoutInternals
  }
    ? keyof DatabaseWithoutInternals[DefaultSchemaEnumNameOr
        Options["schema"]]["Enums"]
    : never = never,
> = DefaultSchemaEnumNameOrOptions extends {
  schema: keyof DatabaseWithoutInternals
}
  ? DatabaseWithoutInternals[DefaultSchemaEnumNameOrOptions[
      "schema"]]["Enums"][EnumName]

  : DefaultSchemaEnumNameOrOptions extends keyof
      DefaultSchema["Enums"]
    ? DefaultSchema["Enums"][DefaultSchemaEnumNameOrOptions]
    : never

export type CompositeTypes<
  PublicCompositeTypeNameOrOptions extends
    | keyof DefaultSchema["CompositeTypes"]
    | { schema: keyof DatabaseWithoutInternals },
  CompositeTypeName extends PublicCompositeTypeNameOrOptions
      extends {
    schema: keyof DatabaseWithoutInternals
  }
    ? keyof DatabaseWithoutInternals[PublicCompositeTypeName
        OrOptions["schema"]]["CompositeTypes"]
    : never = never,
> = PublicCompositeTypeNameOrOptions extends {
  schema: keyof DatabaseWithoutInternals
}
  ? DatabaseWithoutInternals[PublicCompositeTypeNameOrOption
      s["schema"]]["CompositeTypes"][CompositeTypeName]
  : PublicCompositeTypeNameOrOptions extends keyof
      DefaultSchema["CompositeTypes"]
    ? DefaultSchema["CompositeTypes"][PublicCompositeTypeNam
        eOrOptions]
    : never

export const Constants = {
  public: {
    Enums: {
      app_role: ["admin", "user"],
    },
  },
} as const
```

Result Output:

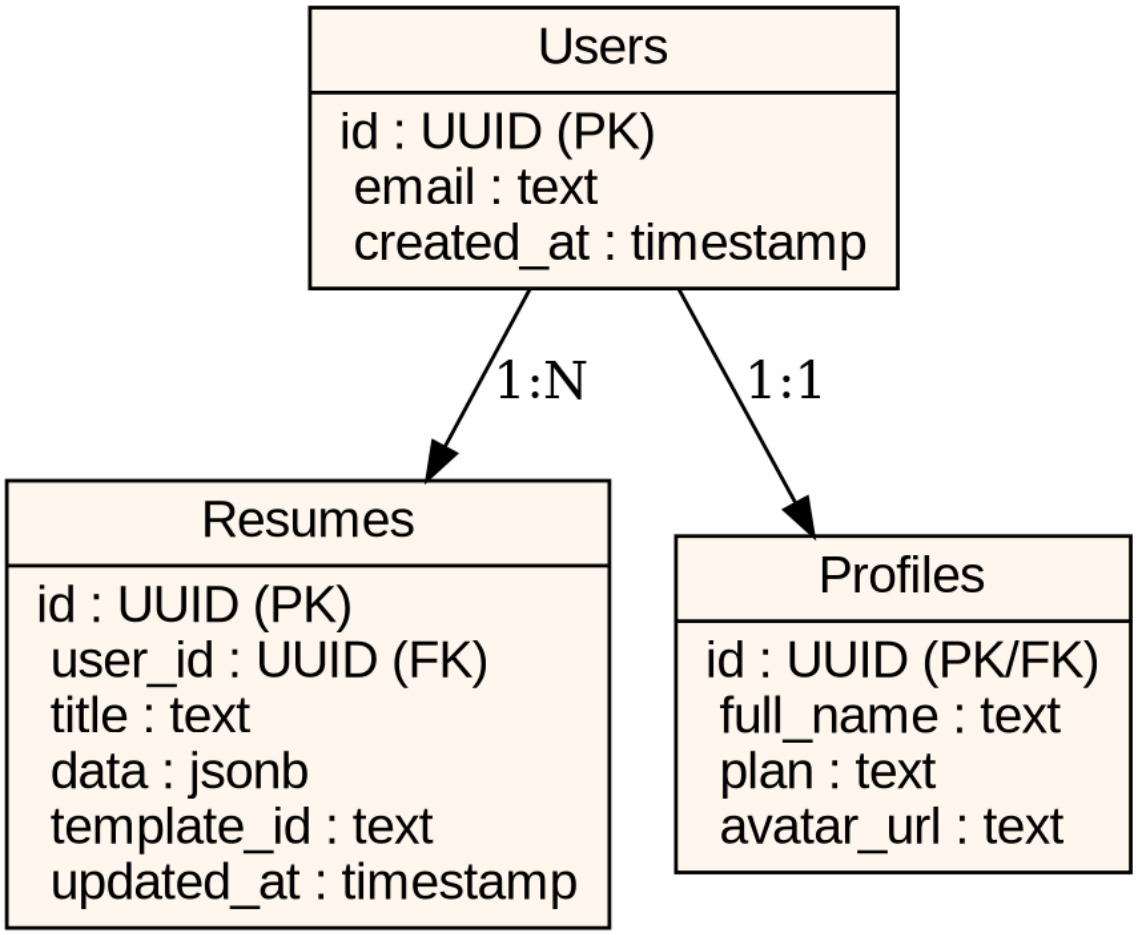

Figure 2: Practical System Output

Practical No. 3

Aim: To develop a state-driven resume builder with dynamic section management.

Source Code Implementation:

```
import { useMemo, useState, useEffect } from "react";
import { useParams } from "react-router-dom";
import { ArrowDown, ArrowUp, Award, Briefcase, FolderKanban,
    GraduationCap, Languages, Plus, Trophy, Trash2,
    UserRound } from "lucide-react";

import { Button } from "@/components/ui/button";
import { Card, CardContent, CardHeader, CardTitle } from
    "@/components/ui/card";
import { Input } from "@/components/ui/input";
import { Textarea } from "@/components/ui/textarea";
import { Switch } from "@/components/ui/switch";
import { Separator } from "@/components/ui/separator";
import { supabase } from "@/integrations/supabase/client";
import { useToast } from "@/hooks/use-toast";
import { Link } from "react-router-dom";
import { EmptyStateCard } from
    "@/components/resume/EmptyStateCard";
import { StepProgressHeader } from
    "@/components/resume/StepProgressHeader";
import { SaveIndicator } from
    "@/components/resume/SaveIndicator";
import { useResumes } from "@/hooks/useResumes";
import { useAutoSave } from "@/hooks/useAutoSave";
import { bumpAiUsageMetric, bumpResumeCreatedMetric,
    bumpTemplateUsageMetric, getActiveApiKey,
    isTemplateVisible } from "@/lib/demoStorage";

type Education = { school: string; degree: string; year:
    string };
type Experience = { company: string; role: string; bullets:
    string };
type Project = { name: string; bullets: string };
type Certification = { name: string; org: string; year:
    string };

type StepId =
  | "profile"
  | "education"
  | "projects"
  | "skills"
  | "languages"
  | "achievements"
```

```
  | "experience"
  | "certs"
  | "templates"
  | "preview";

export default function ResumeBuilder() {
  const { id } = useParams<{ id: string }>();
  const { getResume } = useResumes();
  const { toast } = useToast();

  const [step, setStep] = useState<StepId>("profile");
  const [unlocked, setUnlocked] = useState<Record<StepId,
      boolean>>({
    profile: true,

    education: false,
    projects: false,
    skills: false,
    languages: false,
    achievements: false,
    experience: false,
    certs: false,
    templates: false,
    preview: false,
  });

  const [name, setName] = useState("");
  const [headline, setHeadline] = useState("");
  const [email, setEmail] = useState("");
  const [phone, setPhone] = useState("");
  const [summary, setSummary] = useState("");
  const [summaryAiInput, setSummaryAiInput] = useState("");
  const [photoDataUrl, setPhotoDataUrl] =
      useState<string>("");
  const [skills, setSkills] = useState<string>("");
  const [languages, setLanguages] = useState<string>("");
  const [achievements, setAchievements] =
      useState<string>("");

  const [aiBusy, setAiBusy] = useState<string | null>(null);

  const [showEducation, setShowEducation] = useState(true);
  const [showProjects, setShowProjects] = useState(true);
  const [showSkills, setShowSkills] = useState(true);
  const [showLanguages, setShowLanguages] = useState(true);
  const [showAchievements, setShowAchievements] =
      useState(true);
  const [showExperience, setShowExperience] =
      useState(true);
```

```
const [showCerts, setShowCerts] = useState(true);

const [education, setEducation] =
    useState<Education[]>([]);
const [projects, setProjects] = useState<Project[]>([]);
const [experience, setExperience] =
    useState<Experience[]>([]);
const [certs, setCerts] = useState<Certification[]>([]);

const [selectedTemplate, setSelectedTemplate] =
    useState<string>("classic");

const templates = useMemo(
  () =>
    [
      // Normal templates - unique designs
      {
        id: "classic", name: "Classic", kind: "normal" as
            const,

        note: "Traditional ATS-safe layout with center
            alignment",
        design: "classic" as const
      },
      {
        id: "minimal", name: "Minimal", kind: "normal" as
            const,
        note: "Clean modern with extra white space",
        design: "minimal" as const
      },
      {
        id: "modern", name: "Modern", kind: "normal" as
            const,
        note: "Bold headers with section dividers",
        design: "modern" as const
      },
      {
        id: "executive", name: "Executive", kind: "normal"
            as const,
        note: "Professional with strong hierarchy",
        design: "executive" as const
      },
      {
        id: "twocol", name: "Two-Column", kind: "normal"
            as const,
        note: "Left sidebar for skills, right for
            content",
        design: "twocol" as const
      },
```

```
{
  id: "compact", name: "Compact", kind: "normal" as
      const,
  note: "Fits more content in less space",
  design: "compact" as const
},
{
  id: "atspro", name: "ATS Pro", kind: "normal" as
      const,
  note: "Optimized for resume scanners",
  design: "atspro" as const
},
{
  id: "slate", name: "Slate", kind: "normal" as
      const,
  note: "Soft gray accents with clean look",
  design: "slate" as const
},
{
  id: "nimbus", name: "Nimbus", kind: "normal" as
      const,
  note: "Light separators and subtle styling",
  design: "nimbus" as const
},
{
  id: "vertex", name: "Vertex", kind: "normal" as
      const,

  note: "Sharp geometric section blocks",
  design: "vertex" as const
},
// Color templates - with accent colors
{
  id: "aurora", name: "Aurora", kind: "color" as
      const,
  note: "Purple gradient header bar",
  design: "aurora" as const, accent: "#8b5cf6"
},
{
  id: "metro", name: "Metro", kind: "color" as
      const,
  note: "Blue section markers",
  design: "metro" as const, accent: "#3b82f6"
},
{
  id: "nova", name: "Nova", kind: "color" as const,
  note: "Green accent sidebar",
  design: "nova" as const, accent: "#10b981"
},
```

```
      {
        id: "pulse", name: "Pulse", kind: "color" as
            const,
        note: "Pink skills with chip styling",
        design: "pulse" as const, accent: "#ec4899"
      },
      {
        id: "orbit", name: "Orbit", kind: "color" as
            const,
        note: "Orange accent dividers",
        design: "orbit" as const, accent: "#f59e0b"
      },
      {
        id: "colorpop", name: "Color Pop", kind: "color"
            as const,
        note: "Bold red accents",
        design: "colorpop" as const, accent: "#ef4444"
      },
      {
        id: "elegant", name: "Elegant", kind: "color" as
            const,
        note: "Indigo accent with lines",
        design: "elegant" as const, accent: "#6366f1"
      },
      {
        id: "creative", name: "Creative", kind: "color" as
            const,
        note: "Teal modern accent layout",
        design: "creative" as const, accent: "#14b8a6"
      },
      {

        id: "bold", name: "Bold", kind: "color" as const,
        note: "Red high-contrast headings",
        design: "bold" as const, accent: "#dc2626"
      },
      {
        id: "professional", name: "Professional", kind:
            "color" as const,
        note: "Sky blue accent tags",
        design: "professional" as const, accent: "#0ea5e9"
      },
    ],
  []
);

const [order, setOrder] = useState<Array<"education" |
    "projects" | "skills" | "languages" | "achievements" |
    "experience" | "certs">>([
```

```
  "education",
  "projects",
  "skills",
  "languages",
  "achievements",
  "experience",
  "certs",
]);

const sectionEnabled = useMemo(
  () => ({
    education: showEducation,
    projects: showProjects,
    skills: showSkills,
    languages: showLanguages,
    achievements: showAchievements,
    experience: showExperience,
    certs: showCerts,
  }),
  [showEducation, showProjects, showSkills, showLanguages,
      showAchievements, showExperience, showCerts],
);

useEffect(() => {
  if (!id) return;
  const fetchIt = async () => {
    const resume = await getResume(id);
    if (resume) {
      if (resume.data) {
        const d = resume.data;
        setName(d.name || "");
        setHeadline(d.headline || "");

        setEmail(d.email || "");
        setPhone(d.phone || "");
        setSummary(d.summary || "");
        setPhotoDataUrl(d.photoDataUrl || "");
        setSkills(d.skills || "");
        setLanguages(d.languages || "");
        setAchievements(d.achievements || "");
        setEducation(d.education || []);
        setProjects(d.projects || []);
        setExperience(d.experience || []);
        setCerts(d.certs || []);
      }
      if (resume.template_id)
          setSelectedTemplate(resume.template_id);
      if (resume.section_order)
          setOrder(resume.section_order as any);
```

```
      if (resume.section_enabled) {
        const se = resume.section_enabled;
        setShowEducation(se.education ?? true);
        setShowProjects(se.projects ?? true);
        setShowSkills(se.skills ?? true);
        setShowLanguages(se.languages ?? true);
        setShowAchievements(se.achievements ?? true);
        setShowExperience(se.experience ?? true);
        setShowCerts(se.certs ?? true);
      }
    }
  };
  fetchIt();
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, [id]);

const currentData = useMemo(() => ({ name, headline,
    email, phone, summary, photoDataUrl, skills,
    languages, achievements, education, projects,
    experience, certs }), [name, headline, email, phone,
    summary, photoDataUrl, skills, languages,
    achievements, education, projects, experience,
    certs]);
const { isSaving, lastSavedAt } = useAutoSave(id || "",
    currentData, selectedTemplate, order, sectionEnabled);

const move = (id: (typeof order)[number], dir: -1 | 1) =>
    {
  setOrder((prev) => {
    const i = prev.indexOf(id);
    const j = i + dir;
    if (i < 0 || j < 0 || j >= prev.length) return prev;
    const next = [...prev];
    [next[i], next[j]] = [next[j], next[i]];
    return next;
  });
};

const visibleTemplates = useMemo(() =>
    templates.filter((t) => isTemplateVisible(t.id)),
    [templates]);

const SectionHeader = ({
  title,
  enabled,
  onEnabledChange,
  onMoveUp,
  onMoveDown,
}: {
```

```
  title: string;
  enabled: boolean;
  onEnabledChange: (v: boolean) => void;
  onMoveUp: () => void;
  onMoveDown: () => void;
}) => (
  <div className="flex flex-wrap items-center justify-
      between gap-3">
    <div>
      <p className="font-medium">{title}</p>
      <p className="text-sm text-muted-foreground">Toggle
          on/off and reorder for preview.</p>
    </div>
    <div className="flex items-center gap-2">
      <div className="flex items-center gap-2 rounded-md
          border px-3 py-2">
        <p className="text-sm text-muted-
            foreground">Show</p>
        <Switch checked={enabled}
            onCheckedChange={onEnabledChange} />
      </div>
      <Button variant="outline" size="icon"
          onClick={onMoveUp} aria-label="Move section up">
        <ArrowUp className="h-4 w-4" />
      </Button>
      <Button variant="outline" size="icon"
          onClick={onMoveDown} aria-label="Move section
          down">
        <ArrowDown className="h-4 w-4" />
      </Button>
    </div>
  </div>
);

const steps = useMemo(
  () =>
    [
      { id: "profile" as const, label: "Profile" },
      { id: "education" as const, label: "Education" },
      { id: "projects" as const, label: "Projects" },
      { id: "skills" as const, label: "Skills" },
      { id: "languages" as const, label: "Languages" },
      { id: "achievements" as const, label: "Achievements"
          },
      { id: "experience" as const, label: "Experience" },
      { id: "certs" as const, label: "Certifications" },

      { id: "templates" as const, label: "Templates" },
      { id: "preview" as const, label: "Preview" },
```

```
    ],
  [],
);

const currentIndex = steps.findIndex((s) => s.id ===
    step);
const goTo = (id: StepId) => {
  if (!unlocked[id]) return;
  setStep(id);
};
const unlockAndGoNext = () => {
  const next = steps[currentIndex + 1]?.id;
  if (!next) return;
  setUnlocked((p) => ({ ...p, [next]: true }));
  setStep(next);
};
const goBack = () => {
  const prev = steps[currentIndex - 1]?.id;
  if (!prev) return;
  setStep(prev);
};

const handlePhoto = async (file?: File | null) => {
  if (!file) return;
  if (!file.type.startsWith("image/")) {
    toast({ variant: "destructive", title: "Invalid file",
        description: "Please select an image." });
    return;
  }
  const reader = new FileReader();
  reader.onload = () =>
      setPhotoDataUrl(String(reader.result || ""));
  reader.readAsDataURL(file);
};

const aiGenerate = async ({
  key,
  prompt,
  onApply,
}: {
  key: string;
  prompt: string;
  onApply: (text: string) => void;
}) => {
  setAiBusy(key);

  const activeKey = getActiveApiKey();
  const payload = {
    model: "anthropic/claude-3-opus",
```

```
  messages: [
    { role: "system", content: "You are an expert ATS
        resume writer. CRITICAL: Strictly adhere ONLY to
        the facts provided in the prompt. DO NOT invent
        fake metrics, companies, or experiences. Focus
        on professional phrasing and ATS optimization
        while remaining 100% honest to the user's input.
        Output ONLY the generated text, no chat or
        filler." },
    { role: "user", content: prompt }
  ],
  max_tokens: 250,
  temperature: 0.5,
};

try {
  const response = await
      fetch("https://openrouter.ai/api/v1/chat/completio
      ns", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "Authorization": `Bearer ${activeKey}`,
      "HTTP-Referer": "https://ai-resume-studio.com",
      "X-Title": "AI Resume Studio",
    },
    body: JSON.stringify(payload),
  });

  if (!response.ok) {
    throw new Error(`OpenRouter Error:
        ${response.status}`);
  }

  const data = await response.json();
  let content =
      String(data.choices?.[0]?.message?.content ||
      "").trim();

  if (!content) {
    throw new Error("Empty Response");
  }
  onApply(content);
  bumpAiUsageMetric();
  toast({ title: "AI generated", description: "You can
      edit it manually too." });
} catch (e) {
  console.warn("AI Generate failed, falling back to mock
```

```
        text:", e);
    const mockContent = key.startsWith("project")
      ? "• Developed scalable components using modern
          frameworks\n• Reduced application load time by
          optimizing assets\n• Collaborated closely with
          designers to ensure responsive rendering"
      : key === "summary"
      ? "Results-oriented professional with a strong
          foundation in modern development practices.
          Skilled in building responsive and accessible
          interfaces while collaborating effectively with
          cross-functional teams."
      : key === "skills"
      ? "JavaScript, TypeScript, React, Node.js, HTML,
          CSS, Git, Tailwind"
      : "• Implemented best practices resulting in 20%
          performance increase\n• Maintained code quality
          through peer reviews and strong testing";

    onApply(mockContent);
    bumpAiUsageMetric();
    toast({ title: "AI generated (Mock)", description:
        "You can edit it manually too." });
  } finally {
    setAiBusy(null);
  }
};

return (
  <div className="mx-auto w-full max-w-6xl">
    <div className="flex flex-col sm:flex-row sm:items-
        center sm:justify-between gap-4 mb-4">
      <StepProgressHeader
        title="Resume Builder"
        stepLabel={`Step ${currentIndex + 1} of
            ${steps.length}`}
        stepText={steps[currentIndex]?.label ?? ""}
        progress={(currentIndex + 1) / steps.length}
      />
      <div className="shrink-0 mb-4 sm:mb-0 bg-card
          rounded-md px-3 py-2 border w-fit">
        <SaveIndicator isSaving={isSaving}
            lastSavedAt={lastSavedAt} />
      </div>
    </div>

    {/* Optional quick jump chips (hidden on mobile to
        match reference UI) */}
    <div className="mt-4 hidden flex-wrap gap-2 sm:flex">
```

```
  {steps.map((s, i) => {
    const active = s.id === step;
    const canOpen = unlocked[s.id];
    return (
      <button
        key={s.id}
        type="button"
        onClick={() => goTo(s.id)}
        disabled={!canOpen}
        className={
          "inline-flex items-center gap-2 rounded-full
              border px-3 py-1.5 text-sm transition "
              +
          (active
            ? "bg-muted text-foreground"
            : canOpen
              ? "bg-background/40 text-muted-
                  foreground hover:bg-muted/50"
              : "bg-background/30 text-muted-
                  foreground/60 opacity-60")
        }
      >
        <span className="text-xs font-
            semibold">{String(i + 1).padStart(2,
            "0")}</span>
        <span className="font-medium">{s.label}</span>

      </button>
    );
  })}
</div>

<div className="mt-6 grid gap-6 lg:grid-cols-2">
  {/* Editor */}
  <div className="grid gap-6">
    {step === "profile" ? (
      <Card>
        <CardHeader>
          <CardTitle className="text-lg">Personal
              Information</CardTitle>
        </CardHeader>
        <CardContent className="grid gap-4">
          <p className="text-sm text-muted-
              foreground">Let's start with your basic
              details. This information appears at the
              top of your resume.</p>

          <div className="rounded-xl border border-
              dashed bg-muted/20 p-5">
```

```
  <div className="grid place-items-center
      gap-3 text-center">
    <div className="grid h-16 w-16 place-
        items-center rounded-full border bg-
        background">
      {photoDataUrl ? (
        <img src={photoDataUrl}
            alt="Profile" className="h-16
            w-16 rounded-full object-cover"
            />
      ) : (
        <UserRound className="h-7 w-7 text-
            muted-foreground" />
      )}
    </div>
    <div className="grid gap-1">
      <Button variant="outline" asChild>
        <label className="cursor-pointer">
          Upload Photo
          <input
            type="file"
            accept="image/*"
            onChange={(e) =>
                handlePhoto(e.target.files?.
                [0])}
            className="sr-only"
          />
        </label>
      </Button>
      <p className="text-xs text-muted-
          foreground">Optional · Max 5MB
          (JPG, PNG)</p>
    </div>
  </div>
</div>

<Input placeholder="Full Name" value={name}
    onChange={(e) =>
    setName(e.target.value)} />
<Input
  placeholder="Headline (e.g., BCA Student |
      Frontend Developer)"

  value={headline}
  onChange={(e) =>
      setHeadline(e.target.value)}
/>
<div className="grid gap-3 sm:grid-cols-2">
  <Input placeholder="Email" value={email}
```

```
        onChange={(e) =>
        setEmail(e.target.value)} />
    <Input placeholder="Phone" value={phone}
        onChange={(e) =>
        setPhone(e.target.value)} />
  </div>

  <div className="grid gap-2">
    <p className="text-sm font-
        medium">Professional Summary</p>
    <p className="text-sm text-muted-
        foreground">
      Optional: Generate with AI above, or
          write manually below.
    </p>

    {/* AI box */}
    <div className="grid gap-2 rounded-xl
        border bg-muted/20 p-3">
      <div className="flex flex-wrap items-
          center justify-between gap-2">
        <p className="text-sm font-
            medium">Generate with AI</p>
        <Button
          variant="outline"
          size="sm"
          disabled={aiBusy === "summary"}
          onClick={() =>
            aiGenerate({
              key: "summary",
              prompt:
                summaryAiInput.trim() ||
                `Write a professional resume
                    summary in 3–4 lines for:
                    ${name || "a candidate"}.
                    Headline: ${headline}.`,
              onApply: (t) => setSummary(t),
            })
          }
        >
          {aiBusy === "summary" ?
              "Generating..." : "Generate"}
        </Button>
      </div>
      <Textarea
        placeholder="Enter details for AI
            (role, skills, goals). Example:
            BCA student, frontend, HTML CSS
```

```
                JS, fresher, ATS friendly."
              value={summaryAiInput}
              onChange={(e) =>
                  setSummaryAiInput(e.target.value)}
              className="min-h-[90px]"
            />
            <p className="text-xs text-muted-
                foreground">Generated content will
                appear in the box below for
                editing.</p>
          </div>

          {/* Manual box */}

          <div className="grid gap-2">
            <p className="text-sm font-medium">Write
                manually</p>
            <Textarea
              placeholder="Professional Summary (2–4
                  lines)"
              value={summary}
              onChange={(e) =>
                  setSummary(e.target.value)}
              className="min-h-[120px]"
            />
            <p className="text-xs text-muted-
                foreground">Write directly here if
                you prefer manual entry.</p>
          </div>
        </div>
      </CardContent>
    </Card>
  ) : null}

  {step === "education" ? (
    <Card>
      <CardHeader>
        <SectionHeader
          title="Education"
          enabled={showEducation}
          onEnabledChange={setShowEducation}
          onMoveUp={() => move("education", -1)}
          onMoveDown={() => move("education", 1)}
        />
      </CardHeader>
      <CardContent className="grid gap-4">
        {education.length === 0 ? (
          <EmptyStateCard
            title="No education added yet"
```

```
    description="Add your educational
        qualifications to strengthen your
        resume."
    icon={<GraduationCap className="h-7 w-7
        text-muted-foreground" />}
    actionLabel="Add Education"
    onAction={() => setEducation([{ school:
        "", degree: "", year: "" }])}
  />
) : (
  <>
    {education.map((ed, idx) => (
      <div key={idx} className="rounded-xl
          border bg-card p-4">
        <div className="flex items-center
            justify-between gap-3">
          <p className="text-sm font-
              medium">Education {idx +
              1}</p>
          <Button
            variant="outline"
            size="icon"
            onClick={() => setEducation((p)
                => p.filter((_, i) => i !==
                idx))}

            aria-label="Remove education"
          >
            <Trash2 className="h-4 w-4" />
          </Button>
        </div>
        <div className="mt-3 grid gap-3">
          <Input
            placeholder="Institution Name"
            value={ed.school}
            onChange={(e) =>
              setEducation((p) => p.map((x,
                  i) => (i === idx ? { ...x,
                  school: e.target.value } :
                  x)))
            }
          />
          <Input
            placeholder="Degree"
            value={ed.degree}
            onChange={(e) =>
              setEducation((p) => p.map((x,
                  i) => (i === idx ? { ...x,
                  degree: e.target.value } :
```

```
                        x)))
                    }
                  />
                  <div className="grid gap-3
                      sm:grid-cols-2">
                    <Input
                      placeholder="Start / End Year"
                      value={ed.year}
                      onChange={(e) =>
                        setEducation((p) =>
                            p.map((x, i) => (i ===
                            idx ? { ...x, year:
                            e.target.value } : x)))
                      }
                    />
                    <Input placeholder="GPA
                        (Optional)" />
                  </div>
                </div>
              </div>
            ))}
            <Button
              variant="outline"
              onClick={() => setEducation((p) =>
                  [...p, { school: "", degree: "",
                  year: "" }])}
            >
              <Plus className="mr-2 h-4 w-4" /> Add
                  Another Education
            </Button>
          </>
        )}
      </CardContent>
    </Card>
  ) : null}

  {step === "projects" ? (
    <Card>
      <CardHeader>
        <SectionHeader
          title="Projects"
          enabled={showProjects}
          onEnabledChange={setShowProjects}
          onMoveUp={() => move("projects", -1)}
          onMoveDown={() => move("projects", 1)}
        />
      </CardHeader>
      <CardContent className="grid gap-4">
        {projects.length === 0 ? (
```

```
  <EmptyStateCard
    title="No projects added yet"
    description="Add projects to demonstrate
        your practical skills."
    icon={<FolderKanban className="h-7 w-7
        text-muted-foreground" />}
    actionLabel="Add Project"
    onAction={() => setProjects([{ name: "",
        bullets: "" }])}
  />
) : (
  <>
    {projects.map((p, idx) => (
      <div key={idx} className="rounded-xl
          border bg-card p-4">
        <div className="flex items-center
            justify-between gap-3">
          <p className="text-sm font-
              medium">Project {idx + 1}</p>
          <Button
            variant="outline"
            size="icon"
            onClick={() => setProjects((pp)
                => pp.filter((_, i) => i !==
                idx))}
            aria-label="Remove project"
          >
            <Trash2 className="h-4 w-4" />
          </Button>
        </div>
        <div className="mt-3 grid gap-3">
          <Input
            placeholder="Project Name"
            value={p.name}
            onChange={(e) =>
              setProjects((pp) => pp.map((x,
                  i) => (i === idx ? { ...x,
                  name: e.target.value } :
                  x)))
            }
          />

          <div className="grid gap-2">

            <div className="flex flex-wrap
                items-center justify-between
                gap-2">
              <p className="text-sm font-
                  medium">Description</p>
```

```
                <Button
                  variant="outline"
                  size="sm"
                  disabled={aiBusy ===
                      `project_${idx}`}
                  onClick={() =>
                    aiGenerate({
                      key: `project_${idx}`,
                      prompt: `Write 3–4 lines
                          (no special symbols)
                          as ATS-friendly
                          bullet points (one
                          per line) for a
                          resume project
                          named: ${p.name ||
                          "My Project"}.`,
                      onApply: (t) =>
                        setProjects((pp) =>
                            pp.map((x, i) =>
                            (i === idx ? {
                            ...x, bullets: t }
                            : x))),
                    })
                  }
                >
                  {aiBusy === `project_${idx}`
                      ? "Generating..." : "AI
                      Generate"}
                </Button>
              </div>
              <Textarea
                placeholder="Describe your
                    project, its purpose, and
                    your contributions..."
                value={p.bullets}
                onChange={(e) =>
                  setProjects((pp) =>
                      pp.map((x, i) => (i ===
                      idx ? { ...x, bullets:
                      e.target.value } : x)))
                }
                className="min-h-[130px]"
```

Result Output:

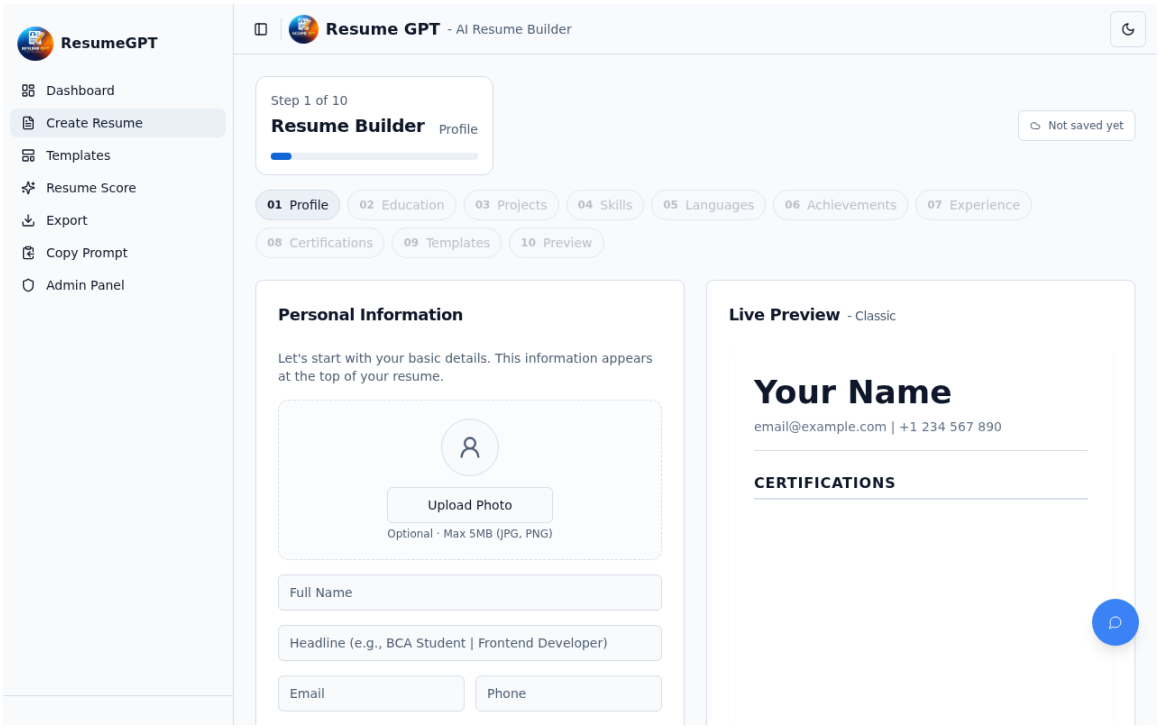

Figure 3: Practical System Output

Practical No. 4

Aim: To integrate Claude-3 Opus via OpenRouter API and implement system prompts for AI content generation.

Source Code Implementation:

```
import { useMemo, useState } from "react";
import { Card, CardContent, CardDescription, CardHeader,
    CardTitle } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Textarea } from "@/components/ui/textarea";

export default function PromptPage() {
  const prompt = useMemo(
    () =>
      `Create a web application called “ResumeGPT —
          Professional Resume Builder” with these
          requirements.

GOAL
Build a resume builder with a dashboard experience and an
    admin panel that controls global AI behavior and API key
    management.

TECH STACK
- React + Vite + TypeScript
- Tailwind + shadcn-ui
- Routing with react-router-dom
- Backend via Supabase (database, auth, server functions)

BRANDING + BASIC INFO (MUST BE VISIBLE AT TOP)
- App name everywhere: ResumeGPT
- Landing page and header must clearly show (immediately,
    not hidden):
  - College: Janaprabha College, Ramtek
  - Class: BCA 3rd Year
  - Group Members: Vansh Khandekar, Shubham Chandekar, Rahul
      Yenurkar, Pranay Mende

ROUTES
- / → Public landing page (academic project report style)
- /dashboard → Dashboard home
- /create → Resume Builder (multi-step)
- /templates → Template picker
- /score → Resume Score (demo)
- /export → Export/Download page
- /admin → Admin Panel (protected)
```

- /prompt → A page that shows THIS prompt and provides a one-click Copy button

PUBLIC LANDING PAGE (/)
- Design the landing page like a professional academic report.
- Hero must show ResumeGPT + short subtitle.
- Show the basic info strip (college/class/members) near the top (always visible).
- Include “Build Resume” CTA linking to /create.
- Include an accordion report section:
  - Sections: Project Details, Abstract, Introduction, Objectives, Process/Workflow, Development Tools, Conclusion
  - Only one section opens at a time
  - Smooth animations and clear expand/collapse affordance

DASHBOARD LAYOUT
- Collapsible sidebar with nav links: Dashboard, Create Resume, Templates, Resume Score, Export, Admin Panel, Copy Prompt
- Sticky top bar with app name + theme toggle
- Main area renders routed pages
- Optional floating AI assistant only inside dashboard

FEATURE: RESUME BUILDER (/create)
- Multi-step flow: profile, education, projects, skills, experience, certifications, preview.
- Each step has:
  - Clear header with current step label + progress bar
  - Add/edit/remove cards per section
  - Validation + friendly toasts
- Live preview panel updates as user edits.
- Allow enabling/disabling sections and reordering them.
- Support optional photo upload preview (client-side).

FEATURE: TEMPLATES (/templates)
- Templates UI only (do not connect templates to resume preview yet).
- Show a grid of 20 templates with clear visual preview thumbnails (normal + color styles).
- Selecting a template highlights it and updates a “Selected Template” card.
- Provide a Reset action to return to default.
- Templates list to include a mix like:
  Classic, Minimal, ATS Pro, Modern, Executive, Serif, Compact, Mono, Timeline, Split
  Aurora, Nova, Pulse, Coral, Emerald, Sunset, Indigo, Citrus, Ocean, Rose

FEATURE: RESUME SCORE (/score)
- Show demo scoring widgets (overall + ATS compatibility) with progress bars.

FEATURE: EXPORT (/export)
- Let user choose Manual vs AI Enhanced preview.
- Provide a single “Download PDF (A4)” action that downloads a PDF directly.
- CRITICAL: Do NOT open the browser print dialog. Do NOT use window.print().
- The user experience must be: click download → file saves as .pdf immediately.

AI ASSISTANT (USER-FACING)
- Provide an assistant UI (floating) where user types a prompt and receives a short ATS-friendly suggestion.
- Must call a backend function (server function) to generate text.
- Error handling:
  - rate limit → show toast
  - quota exceeded → show toast
  - generic error → show toast

STRICT AI OUTPUT RULES (ENFORCE SERVER-SIDE)
- Output must be 6–10 short lines.

- No bullet points, numbering, markdown, emojis.
- Sanitize and strip problematic characters and normalize whitespace.
- Keep it ATS-friendly and professional.

ADMIN PANEL (/admin)
- Email/password authentication.
- Protect access via RBAC:
  - Store roles in database table user_roles.
  - Provide server-side RPC/function is_admin() to verify.
- Include a one-time “Admin Bootstrap” action:
  - If no admin exists yet, the logged-in user can call a backend function to become the first admin.
  - If an admin already exists, bootstrap must be blocked.

ADMIN: AI SETTINGS
- Database table admin_settings (single row id=1) containing:
  - ai_enabled (boolean)
  - ai_model (string)
  - max_lines (number)
- Admin UI lets you toggle AI, pick model, and set max lines.

```
ADMIN: API KEYS MANAGEMENT
- Database table gemini_api_keys containing:
  - label, api_key, priority, is_active
- Admin can add keys, toggle active, and delete keys.

SECURITY
- Enable row-level security for admin tables.
- Only admins can read/update admin_settings and
    gemini_api_keys.
- Never expose secret keys to non-admin users.

UX
- Modern clean dashboard UI.
- Consistent spacing, cards, and helpful empty states.
- Theme toggle (light/dark).

DELIVERABLE
Implement the full UI, routes, backend tables, RLS policies,
    server functions, and error handling described above.

IMPORTANT CONSTRAINTS
- The /prompt page must only show this plain-text prompt and
    a one-click Copy button. Do not show code on /prompt.
- Do not include any source code in the /prompt page
    content.
`,
    [],
  );

  const [copied, setCopied] = useState(false);

  const copy = async () => {
    await navigator.clipboard.writeText(prompt);
    setCopied(true);
    window.setTimeout(() => setCopied(false), 1200);
  };

  return (
    <div className="mx-auto w-full max-w-4xl">
      <h1 className="text-2xl font-semibold tracking-
          tight">System Prompt</h1>
      <p className="mt-1 text-muted-foreground">Copy the
          master prompt used for this application.</p>

      <Card className="mt-6">
        <CardHeader className="flex flex-row items-start
            justify-between gap-4">
          <div className="grid gap-1">
            <CardTitle>Master Prompt</CardTitle>
```

```
              <CardDescription>Use this prompt to recreate or
                  extend the application
                  logic.</CardDescription>
            </div>
            <Button onClick={copy} className="shrink-0">
              {copied ? "Copied" : "Copy"}
            </Button>
          </CardHeader>
          <CardContent>
            <Textarea value={prompt} readOnly
                classNameName="min-h-[420px] font-mono text-xs" />
          </CardContent>
        </Card>
      </div>
    );
  }
```

Result Output:

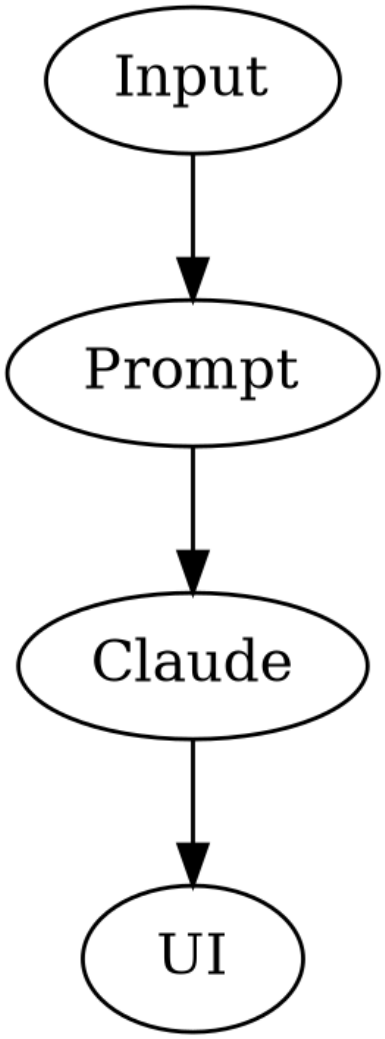

Figure 4: Practical System Output

Practical No. 5

Aim: To implement a rule-based engine for evaluating keyword density and ATS scores.

Source Code Implementation:

```
import { useEffect, useMemo, useState } from "react";
import { AlertCircle, CheckCircle, Loader2, RefreshCw,
    XCircle } from "lucide-react";

import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Card, CardContent, CardDescription, CardHeader,
    CardTitle } from "@/components/ui/card";
import { Progress } from "@/components/ui/progress";
import { useToast } from "@/hooks/use-toast";
import { supabase } from "@/integrations/supabase/client";
import { bumpAiUsageMetric, getActiveApiKey } from
    "@/lib/demoStorage";

type ResumeData = {
  name: string;
  headline: string;
  email: string;
  phone: string;
  summary: string;
  photoDataUrl: string;
  skills: string;
  languages: string;
  achievements: string;
  education: { school: string; degree: string; year: string
      }[];
  projects: { name: string; bullets: string }[];
  experience: { company: string; role: string; bullets:
      string }[];
  certs: { name: string; org: string; year: string }[];
  selectedTemplate: string;
  order: string[];
  sectionEnabled: Record<string, boolean>;
};

type SectionId = "profile" | "education" | "skills" |
    "experience" | "projects" | "certifications";
type SectionStatus = "good" | "medium" | "missing";

type SectionResult = {
  id: SectionId;
  name: string;
  score: number;
```

```
  status: SectionStatus;
  reason: string;
};

type ScoreResult = {
  overallScore: number;
  atsScore: number;
  sections: SectionResult[];

  improvements: string[];
  summary: string;
  source: "rule" | "ai";
};

type AiSection = {
  id?: string;
  score?: number;
  reason?: string;
};

type AiScorePayload = {
  overallScore?: number;
  atsScore?: number;
  summary?: string;
  improvements?: string[];
  sections?: AiSection[];
};

const emptyResume: ResumeData = {
  name: "",
  headline: "",
  email: "",
  phone: "",
  summary: "",
  photoDataUrl: "",
  skills: "",
  languages: "",
  achievements: "",
  education: [],
  projects: [],
  experience: [],
  certs: [],
  selectedTemplate: "classic",
  order: ["education", "projects", "skills", "experience",
      "certs"],
  sectionEnabled: { education: true, projects: true, skills:
      true, experience: true, certs: true },
};
```

```
const scoreToStatus = (score: number): SectionStatus => {
  if (score <= 10) return "missing";
  if (score < 70) return "medium";
  return "good";
};

const clamp = (value: number, min = 0, max = 100) =>
    Math.max(min, Math.min(max, Math.round(value)));

function computeRuleBasedScore(resume: ResumeData):
    ScoreResult {
  const skillsList = resume.skills
    .split(/[\n,]/)
    .map((s) => s.trim())
    .filter(Boolean);

  const experienceLines = resume.experience
    .flatMap((x) => x.bullets.split("\n"))
    .map((b) => b.trim())
    .filter(Boolean);

  const projectLines = resume.projects
    .flatMap((x) => x.bullets.split("\n"))
    .map((b) => b.trim())
    .filter(Boolean);
  const achievementLines = (resume.achievements || "")
    .split("\n")
    .map((b) => b.trim())
    .filter(Boolean);

  const allAchievementLines = [...experienceLines,
      ...projectLines, ...achievementLines];
  const numberMentions = allAchievementLines.filter((line)
      => /\b\d+(\+|%|x)?\b/i.test(line)).length;
  const actionVerbMentions =
      allAchievementLines.filter((line) =>
    /^(built|developed|implemented|created|designed|improved
        |optimized|led|managed|delivered|reduced|increased|a
        utomated)\b/i.test(
      line,
    ),
  ).length;

  const profileScore = clamp(
    (resume.name ? 25 : 0) +
      (resume.headline ? 25 : 0) +
      (resume.email ? 20 : 0) +
      (resume.phone ? 20 : 0) +
      (resume.summary.length >= 60 ? 10 : 0),
```

```
);

const educationScore = clamp(
  resume.education.length === 0
    ? 0
    : 30 +
        resume.education.reduce((acc, edu) => {
          const complete = Number(Boolean(edu.school)) +
              Number(Boolean(edu.degree)) +
              Number(Boolean(edu.year));
          return acc + complete * 8;
        }, 0),

);

const skillsScore = clamp(
  (skillsList.length === 0 ? 0 : 20) +
    Math.min(skillsList.length * 7, 42) +
    (skillsList.length >= 8 ? 18 : 0) +
    (skillsList.length >= 12 ? 10 : 0),
);

const experienceScore = clamp(
  (resume.experience.length === 0 ? 0 : 24) +
    Math.min(resume.experience.length * 12, 24) +
    Math.min(experienceLines.length * 3, 24) +
    Math.min(numberMentions * 4, 16) +
    Math.min(actionVerbMentions * 2, 12),
);

const projectsScore = clamp(
  (resume.projects.length === 0 ? 0 : 20) +
    Math.min(resume.projects.length * 12, 24) +
    Math.min(projectLines.length * 3, 28) +
    Math.min(projectLines.filter((line) => /\b(react|node|
        python|sql|api|typescript|tailwind|vite|java)\b/i.
        test(line)).length * 4, 20),
);

const certificationsScore = clamp(
  (resume.certs.length === 0 ? 0 : 25) +
    resume.certs.reduce((acc, cert) => {
      const complete = Number(Boolean(cert.name)) +
          Number(Boolean(cert.org)) +
          Number(Boolean(cert.year));
      return acc + complete * 10;
    }, 0),
);
```

```
const sections: SectionResult[] = [
  {
    id: "profile",
    name: "Profile",
    score: profileScore,
    status: scoreToStatus(profileScore),
    reason: profileScore >= 75 ? "Profile details look
        complete." : "Add complete contact details and a
        stronger headline.",
  },
  {
    id: "education",
    name: "Education",
    score: educationScore,
    status: scoreToStatus(educationScore),

    reason: educationScore >= 75 ? "Education section is
        properly filled." : "Add school, degree and year
        for each education entry.",
  },
  {
    id: "skills",
    name: "Skills",
    score: skillsScore,
    status: scoreToStatus(skillsScore),
    reason: skillsScore >= 75 ? "Skills list is relevant
        and detailed." : "Add more role-specific technical
        skills.",
  },
  {
    id: "experience",
    name: "Experience",
    score: experienceScore,
    status: scoreToStatus(experienceScore),
    reason:
      experienceScore >= 75
        ? "Experience has good depth and impact."
        : "Add measurable achievements and action-oriented
            bullets in experience.",
  },
  {
    id: "projects",
    name: "Projects",
    score: projectsScore,
    status: scoreToStatus(projectsScore),
    reason: projectsScore >= 75 ? "Projects demonstrate
        strong practical work." : "Add project outcomes,
        tech stack, and contribution details.",
  },
```

```
  {
    id: "certifications",
    name: "Certifications",
    score: certificationsScore,
    status: scoreToStatus(certificationsScore),
    reason: certificationsScore >= 75 ? "Certifications
        add strong credibility." : "Add relevant
        certifications to improve trust and ATS
        strength.",
  },
];

const overallScore = clamp(
  profileScore * 0.2 +
    educationScore * 0.15 +
    skillsScore * 0.2 +
    experienceScore * 0.2 +
    projectsScore * 0.17 +
    certificationsScore * 0.08,
);

const atsScore = clamp(

  30 +
    Math.min(skillsList.length * 3, 20) +
    Math.min(numberMentions * 5, 20) +
    Math.min(actionVerbMentions * 2, 12) +
    (resume.summary.length >= 80 ? 8 : 0) +
    (resume.experience.length > 0 ? 6 : 0) +
    (resume.projects.length > 0 ? 4 : 0),
);

const improvements: string[] = [];
if (profileScore < 75) improvements.push("Complete profile
    with headline, email, phone, and a concise
    professional summary.");
if (skillsScore < 75) improvements.push("Add 8 to 12 role-
    specific skills including tools and technologies.");
if (experienceScore < 75) improvements.push("Write
    experience bullets with action verbs and measurable
    outcomes.");
if (projectsScore < 75) improvements.push("Mention project
    impact and stack used for each project.");
if (certificationsScore < 50) improvements.push("Include
    certifications relevant to your target role.");
if (improvements.length === 0) improvements.push("Resume
    structure is strong; now tailor keywords to each job
    description.");
```

```
  return {
    overallScore,
    atsScore,
    sections,
    improvements: improvements.slice(0, 5),
    summary:
      overallScore >= 80
        ? "Your resume is strong and close to recruiter-
            ready."
        : overallScore >= 60
          ? "Your resume is good but needs a few targeted
              improvements."
          : "Your resume needs stronger content depth to
              compete effectively.",
    source: "rule",
  };
}

export default function ResumeScore() {
  const [resumeData, setResumeData] =
      useState<ResumeData>(emptyResume);
  const [scoreResult, setScoreResult] =
      useState<ScoreResult>(() =>
      computeRuleBasedScore(emptyResume));
  const [aiLoading, setAiLoading] = useState(false);
  const [hasResume, setHasResume] = useState(false);
  const [lastAnalyzedAt, setLastAnalyzedAt] =
      useState<string>("");
  const { toast } = useToast();

  useEffect(() => {
    const raw = localStorage.getItem("resumeData");
    if (!raw) {
      setHasResume(false);
      setScoreResult(computeRuleBasedScore(emptyResume));

      return;
    }

    try {
      const parsed = JSON.parse(raw) as ResumeData;
      setResumeData(parsed);
      setHasResume(true);
      setScoreResult(computeRuleBasedScore(parsed));
    } catch (error) {
      console.error("Resume data parse error:", error);
      setHasResume(false);
      setScoreResult(computeRuleBasedScore(emptyResume));
    }
```

```
}, []);

const statusIcon = (status: SectionStatus) => {
  if (status === "good") return <CheckCircle
      className="h-4 w-4 text-emerald-600" />;
  if (status === "medium") return <AlertCircle
      className="h-4 w-4 text-amber-600" />;
  return <XCircle className="h-4 w-4 text-red-600" />;
};

const statusClasses = (status: SectionStatus) => {
  if (status === "good") return "border-emerald-200 bg-
      emerald-50 dark:bg-emerald-950/20";
  if (status === "medium") return "border-amber-200 bg-
      amber-50 dark:bg-amber-950/20";
  return "border-red-200 bg-red-50 dark:bg-red-950/20";
};

const scoreMessage = useMemo(() => {
  const score = scoreResult.overallScore;
  if (score >= 85) return "Excellent resume quality. Minor
      refinements can make it outstanding.";
  if (score >= 70) return "Good resume quality. Improve
      weak sections for stronger impact.";
  if (score >= 55) return "Average quality. Focus on
      achievements and project depth.";
  return "Needs strong improvement. Build complete
      sections with measurable outcomes.";
}, [scoreResult.overallScore]);

const runAiAnalysis = async () => {
  if (!hasResume) {
    toast({
      variant: "destructive",
      title: "No resume data found",
      description: "Please fill your resume in Create
          Resume first.",
    });
    return;
  }

  setAiLoading(true);
  const baseline = computeRuleBasedScore(resumeData);

  try {
    const activeKey = getActiveApiKey();
    const payload = {
      model: "anthropic/claude-3-opus",
      messages: [
```

```
    {
      role: "system",
      content: `You are an expert resume grader.
          Analyze the provided resume JSON and
          baseline score. Return a JSON object with:
      - overallScore (0-100)
      - atsScore (0-100)
      - summary (brief overview)
      - improvements (array of 4-6 strings)
      - sections (array of {id: string, score: number,
          reason: string} for profile, education,
          skills, experience, projects,
          certifications).
      Output ONLY the valid JSON.`
    },
    { role: "user", content: `Resume Data:
        ${JSON.stringify(resumeData)}\nBaseline:
        ${JSON.stringify(baseline)}` }
  ],
  response_format: { type: "json_object" },
  max_tokens: 1000,
  temperature: 0.1,
};

let aiData: AiScorePayload = {};
try {
  const response = await
      fetch("https://openrouter.ai/api/v1/chat/complet
      ions", {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
      "Authorization": `Bearer ${activeKey}`,
      "HTTP-Referer": "https://ai-resume-studio.com",
      "X-Title": "AI Resume Studio",
    },
    body: JSON.stringify(payload),
  });

  if (!response.ok) throw new Error(`OpenRouter Error:
      ${response.status}`);
  const rawRes = await response.json();
  const data =
      JSON.parse(rawRes.choices?.[0]?.message?.content
      || "{}");
  if (data?.error === "rate_limit") {
    toast({ variant: "destructive", title: "Rate
        Limit", description: data.message });
```

```
    setScoreResult(baseline);
    return;

  }
  if (data?.error === "quota_exceeded") {
    toast({ variant: "destructive", title: "Quota
        Exhausted", description: data.message });
    setScoreResult(baseline);
    return;
  }
  aiData = (data || {}) as AiScorePayload;
} catch (err) {
  // Fallback robust mock generator
  console.warn("API failed, falling back to realistic
      mock:", err);
  aiData = {
    overallScore: clamp(baseline.overallScore - 5),
    atsScore: clamp(baseline.atsScore - 5),
    summary: "AI analysis detected standard patterns
        but recommends more quantifiable metrics.",
    improvements: [
      "Use stronger action verbs.",
      "Consider adding more impact metrics (%, $,
          numbers) to achievements.",
      "Tailor skills specifically to target roles.",
    ],
    sections: baseline.sections.map(s => ({
      id: s.id,
      score: clamp(s.score - 4),
      reason: "Content is good but could be more
          impactful."
    }))
  };
}

const aiSections = Array.isArray(aiData.sections) ?
    aiData.sections : [];
const byId = new Map(aiSections.map((x) =>
    [String(x.id || "").toLowerCase(), x]));

const mergedSections: SectionResult[] =
    baseline.sections.map((section) => {
  const aiSection = byId.get(section.id);
  const aiScore = Number(aiSection?.score);
  const blendedScore = Number.isFinite(aiScore) ?
      clamp(section.score * 0.6 + aiScore * 0.4) :
      section.score;
  return {
    ...section,
```

```
        score: blendedScore,
        status: scoreToStatus(blendedScore),
        reason:
          typeof aiSection?.reason === "string" &&
              aiSection.reason.trim().length > 0
            ? aiSection.reason.trim()
            : section.reason,
      };
    });

    const aiOverall = Number(aiData.overallScore);
    const aiAts = Number(aiData.atsScore);
    const mergedOverall = Number.isFinite(aiOverall) ?
        clamp(baseline.overallScore * 0.65 + aiOverall *
        0.35) : baseline.overallScore;
    const mergedAts = Number.isFinite(aiAts) ?
        clamp(baseline.atsScore * 0.65 + aiAts * 0.35) :
        baseline.atsScore;

    const improvements =
        Array.isArray(aiData.improvements)
      ? aiData.improvements.map((tip) =>
          String(tip).trim()).filter(Boolean).slice(0, 6)
      : baseline.improvements;

    const summary = typeof aiData.summary === "string" &&
        aiData.summary.trim() ? aiData.summary.trim() :
        baseline.summary;

    setScoreResult({
      overallScore: mergedOverall,
      atsScore: mergedAts,
      sections: mergedSections,
      improvements: improvements.length ? improvements :
          baseline.improvements,
      summary,
      source: "ai",
    });
    setLastAnalyzedAt(new Date().toLocaleString());
    bumpAiUsageMetric();
    toast({
      title: "AI analysis complete",
      description: "Your resume has been scored with full-
          content review.",
    });
  } catch (err) {
    console.error("AI score error:", err);
    setScoreResult(baseline);
    toast({
```

```
      variant: "destructive",
      title: "AI analysis failed",
      description: "Showing deterministic score based on
          your resume content.",
    });
  } finally {
    setAiLoading(false);
  }
};

return (
  <div className="mx-auto w-full max-w-5xl">
    <div className="flex flex-wrap items-start justify-
        between gap-4">
      <div>
        <h1 className="text-2xl font-semibold tracking-
            tight text-foreground">Resume Score</h1>
        <p className="mt-1 text-muted-foreground">AI +
            rubric analysis of your complete resume.</p>
      </div>

      <Button onClick={runAiAnalysis} disabled={aiLoading}
          className="gap-2">
        {aiLoading ? <Loader2 className="h-4 w-4 animate-
            spin" /> : <RefreshCw className="h-4 w-4" />}
        {aiLoading ? "Analyzing..." : "Analyze with AI"}
      </Button>
    </div>

    {!hasResume && (
      <Card className="mt-6 border-amber-300 bg-amber-50
          dark:bg-amber-950/20">
        <CardContent className="pt-6 text-sm text-
            amber-900 dark:text-amber-200">
          No saved resume found. Create your resume first,
              then come back for full AI scoring.
        </CardContent>
      </Card>
    )}

    <div className="mt-6 grid gap-6 md:grid-cols-2">
      <Card className="bg-card border-2 border-
          emerald-500/30">
        <CardHeader>
          <CardTitle className="flex items-center gap-2
              text-foreground">
            <span className="text-2xl">■</span> Overall
                Score
          </CardTitle>
```

```
      <CardDescription className="text-muted-
          foreground">Quality and completeness
          score</CardDescription>
    </CardHeader>
    <CardContent>
      <div className="flex items-end gap-4">
        <p className="text-6xl font-bold text-
            emerald-
            600">{scoreResult.overallScore}</p>
        <p className="mb-2 text-lg text-muted-
            foreground">/ 100</p>
      </div>
      <Progress value={scoreResult.overallScore}
          className="mt-4 h-3" />
      <p className="mt-2 text-sm font-medium text-
          emerald-700 dark:text-
          emerald-400">{scoreMessage}</p>
    </CardContent>
  </Card>

  <Card className="bg-card border-2 border-
      blue-500/30">
    <CardHeader>
      <CardTitle className="flex items-center gap-2
          text-foreground">
        <span className="text-2xl">■</span> ATS
            Compatibility
      </CardTitle>
      <CardDescription className="text-muted-
          foreground">Parsing and keyword readiness
          score</CardDescription>
    </CardHeader>
    <CardContent>
      <div className="flex items-end gap-4">
        <p className="text-6xl font-bold text-
            blue-600">{scoreResult.atsScore}</p>
        <p className="mb-2 text-lg text-muted-
            foreground">/ 100</p>
      </div>
      <Progress value={scoreResult.atsScore}
          className="mt-4 h-3" />

      <p className="mt-2 text-sm font-medium text-
          blue-700 dark:text-
          blue-400">{scoreResult.summary}</p>
    </CardContent>
  </Card>
</div>
```

```
<Card className="mt-6 bg-card">
  <CardHeader>
    <CardTitle className="flex items-center gap-2
        text-foreground">
      <span className="text-xl">■</span> Section
          Strength Analysis
    </CardTitle>
    <CardDescription className="text-muted-
        foreground">Detailed score per section with
        feedback</CardDescription>
  </CardHeader>
  <CardContent>
    <div className="grid gap-4 md:grid-cols-2">
      {scoreResult.sections.map((section) => (
        <div key={section.id} className={`rounded-lg
            border p-4
            ${statusClasses(section.status)}`}>
          <div className="flex items-center gap-3">
            {statusIcon(section.status)}
            <div className="flex-1">
              <div className="flex items-center
                  justify-between">
                <span className="font-medium text-
                    foreground">{section.name}</span>
                <Badge
                  variant={
                    section.status === "good" ?
                        "default" : section.status ===
                        "medium" ? "outline" :
                        "destructive"
                  }
                >
                  {section.score}%
                </Badge>
              </div>
              <Progress value={section.score}
                  className="mt-2 h-2" />
              <p className="mt-2 text-xs text-muted-
                  foreground">{section.reason}</p>
            </div>
          </div>
        </div>
      ))}
    </div>
  </CardContent>
</Card>

<Card className="mt-6 border-blue-200 bg-card">
```

```
          <CardHeader>
            <CardTitle className="flex items-center gap-2
                text-foreground">
              <span className="text-xl">■</span> Improvements
                  to Apply
            </CardTitle>
            <CardDescription className="text-muted-
                foreground">

              {scoreResult.source === "ai" ? "AI-validated
                  suggestions" : "Rule-based suggestions"}
              {lastAnalyzedAt ? ` • Last analyzed:
                  ${lastAnalyzedAt}` : ""}
            </CardDescription>
          </CardHeader>
          <CardContent className="space-y-2 text-foreground">
            {scoreResult.improvements.map((tip, idx) => (
              <p key={`${tip}-${idx}`} className="text-sm">
                - {tip}
              </p>
            ))}
          </CardContent>
        </Card>
      </div>
    );
  }
```

Result Output:

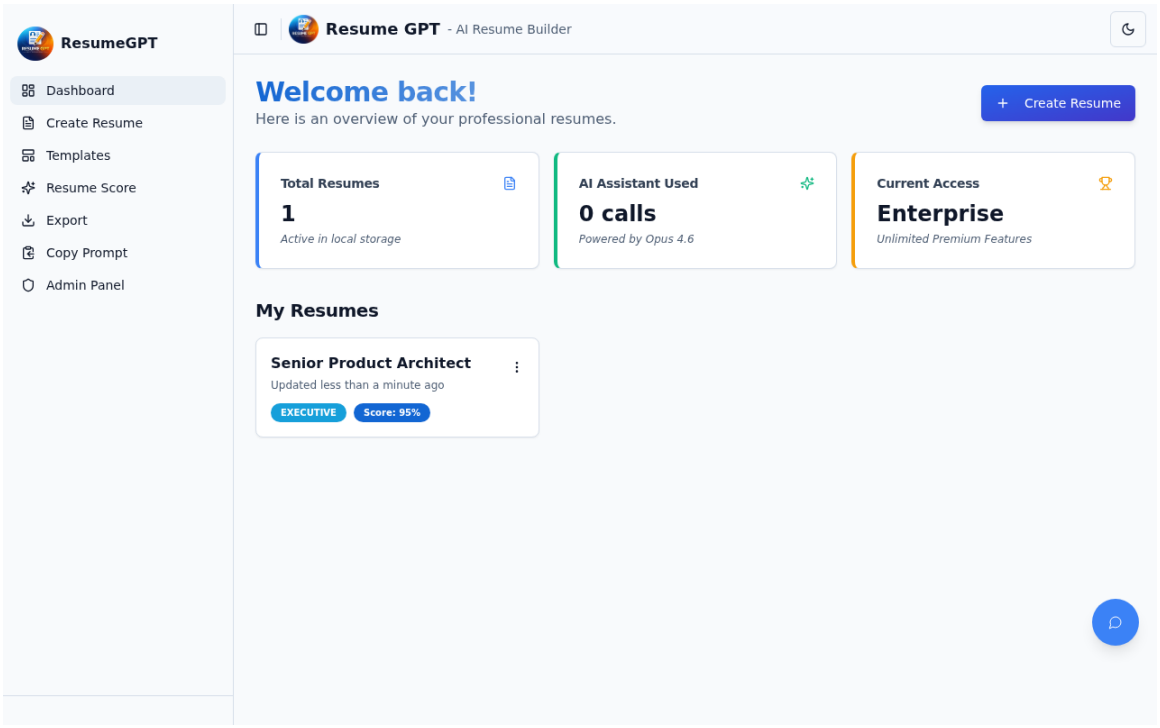

Figure 5: Practical System Output

Practical No. 6

Aim: To implement a browser-side PDF generation engine with high-fidelity layout rendering.

Source Code Implementation:

```
import { useState, useEffect } from "react";
import { Button } from "@/components/ui/button";
import { Card, CardContent, CardDescription, CardHeader,
    CardTitle } from "@/components/ui/card";
import jsPDF from "jspdf";
import { Separator } from "@/components/ui/separator";
import { Link } from "react-router-dom";
import { ArrowLeft } from "lucide-react";

type ResumeData = {
  name: string;
  headline: string;
  email: string;
  phone: string;
  summary: string;
  photoDataUrl: string;
  skills: string;
  languages: string;
  achievements: string;
  education: { school: string; degree: string; year: string
      }[];
  projects: { name: string; bullets: string }[];
  experience: { company: string; role: string; bullets:
      string }[];
  certs: { name: string; org: string; year: string }[];
  selectedTemplate: string;
  order: string[];
  sectionEnabled: Record<string, boolean>;
};

const templates = [
  { id: "classic", name: "Classic", kind: "normal" as const,
      design: "classic", accent: null },
  { id: "minimal", name: "Minimal", kind: "normal" as const,
      design: "minimal", accent: null },
  { id: "modern", name: "Modern", kind: "normal" as const,
      design: "modern", accent: null },
  { id: "executive", name: "Executive", kind: "normal" as
      const, design: "executive", accent: null },
  { id: "twocol", name: "Two-Column", kind: "normal" as
      const, design: "twocol", accent: null },
  { id: "compact", name: "Compact", kind: "normal" as const,
      design: "compact", accent: null },
```

```
  { id: "atspro", name: "ATS Pro", kind: "normal" as const,
      design: "atspro", accent: null },
  { id: "slate", name: "Slate", kind: "normal" as const,
      design: "slate", accent: null },
  { id: "nimbus", name: "Nimbus", kind: "normal" as const,
      design: "nimbus", accent: null },
  { id: "vertex", name: "Vertex", kind: "normal" as const,
      design: "vertex", accent: null },
  { id: "aurora", name: "Aurora", kind: "color" as const,
      design: "aurora", accent: "#8b5cf6" },
  { id: "metro", name: "Metro", kind: "color" as const,
      design: "metro", accent: "#3b82f6" },
  { id: "nova", name: "Nova", kind: "color" as const,
      design: "nova", accent: "#10b981" },
  { id: "pulse", name: "Pulse", kind: "color" as const,
      design: "pulse", accent: "#ec4899" },
  { id: "orbit", name: "Orbit", kind: "color" as const,
      design: "orbit", accent: "#f59e0b" },
  { id: "colorpop", name: "Color Pop", kind: "color" as
      const, design: "colorpop", accent: "#ef4444" },
  { id: "elegant", name: "Elegant", kind: "color" as const,
      design: "elegant", accent: "#6366f1" },

  { id: "creative", name: "Creative", kind: "color" as
      const, design: "creative", accent: "#14b8a6" },
  { id: "bold", name: "Bold", kind: "color" as const,
      design: "bold", accent: "#dc2626" },
  { id: "professional", name: "Professional", kind: "color"
      as const, design: "professional", accent: "#0ea5e9" },
];

export default function ExportResume() {
  const [choice, setChoice] = useState<"manual" |
      "ai">("manual");
  const [resumeData, setResumeData] = useState<ResumeData |
      null>(null);
  const [loaded, setLoaded] = useState(false);

  useEffect(() => {
    const data = localStorage.getItem('resumeData');
    if (data) {
      try {
        setResumeData(JSON.parse(data));
      } catch (e) {
        console.error("Failed to parse resume data", e);
      }
    }
    setLoaded(true);
  }, []);
```

```
const template = templates.find(t => t.id ===
    (resumeData?.selectedTemplate || "classic"));
const isColorTemplate = template?.kind === "color";
const accentColor = template?.accent || null;

const previewData = resumeData || {
  name: "John Doe",
  headline: "Software Developer",
  email: "john@example.com",
  phone: "+1 234 567 8900",
  summary: "Passionate developer with experience in
      building web applications.",
  photoDataUrl: "",
  skills: "JavaScript, React, Node.js, Python",
  languages: "English, Hindi",
  achievements: "",
  education: [{ school: "ABC University", degree: "BCA",
      year: "2024" }],
  projects: [],
  experience: [],
  certs: []
};

const generatePDF = () => {
  const JsConstructor = (jsPDF as any).jsPDF || jsPDF;
  if (typeof JsConstructor !== 'function') {

    console.error("jsPDF library error");
    return;
  }
  const doc = new JsConstructor({ unit: "mm", format: "a4"
      });
  const isAi = choice === "ai";
  const hexToRgb = (hex: string): [number, number, number]
      => {
    const clean = hex.replace("#", "");
    if (clean.length !== 6) return [37, 99, 235];
    return [
      Number.parseInt(clean.slice(0, 2), 16),
      Number.parseInt(clean.slice(2, 4), 16),
      Number.parseInt(clean.slice(4, 6), 16),
    ];
  };

  const pageWidth = doc.internal.pageSize.getWidth();
  const pageHeight = doc.internal.pageSize.getHeight();
  const marginX = 15;
  const contentWidth = pageWidth - marginX * 2;
  const bottomMargin = 14;
```

```
const accentHex = accentColor || (isAi ? "#2563eb" :
    "#1f4b99");
const [accentR, accentG, accentB] = hexToRgb(accentHex);
const [altR, altG, altB] = isAi ? [8, 145, 178] : [29,
    78, 216];

let yPos = 18;

const addPage = () => {
  doc.addPage();
  yPos = 20;
  doc.setDrawColor(226, 232, 240);
  doc.setLineWidth(0.3);
  doc.line(marginX, yPos - 6, pageWidth - marginX, yPos
      - 6);
};

const ensureSpace = (heightNeeded: number) => {
  if (yPos + heightNeeded > pageHeight - bottomMargin) {
    addPage();
  }
};

const drawGradientBar = (
  x: number,
  y: number,
  width: number,

  height: number,
  start: [number, number, number],
  end: [number, number, number],
) => {
  const steps = 80;
  for (let i = 0; i < steps; i += 1) {
    const t = i / (steps - 1);
    const r = Math.round(start[0] + (end[0] - start[0])
        * t);
    const g = Math.round(start[1] + (end[1] - start[1])
        * t);
    const b = Math.round(start[2] + (end[2] - start[2])
        * t);
    doc.setFillColor(r, g, b);
    const segmentW = width / steps;
    doc.rect(x + i * segmentW, y, segmentW + 0.2,
        height, "F");
  }
};

const drawHeader = () => {
```

```
const headerHeight = isAi ? 44 : 34;

if (isAi) {
  drawGradientBar(0, 0, pageWidth, headerHeight,
      [accentR, accentG, accentB], [altR, altG,
      altB]);
} else {
  doc.setFillColor(248, 250, 252);
  doc.rect(0, 0, pageWidth, headerHeight, "F");
  doc.setDrawColor(203, 213, 225);
  doc.setLineWidth(0.5);
  doc.line(0, headerHeight, pageWidth, headerHeight);
}

let textX = marginX;
if (previewData.photoDataUrl) {
  try {
    const photoSize = isAi ? 24 : 20;
    const photoY = isAi ? 9 : 7;
    const format = previewData.photoDataUrl.split(",")
        [0].split("/")[1]?.split(";")[0]?.toUpperCase(
        ) || "JPEG";
    doc.addImage(previewData.photoDataUrl, format as
        any, marginX, photoY, photoSize, photoSize);
    if (isAi) {
      doc.setDrawColor(255, 255, 255);
      doc.setLineWidth(0.8);
    } else {
      doc.setDrawColor(148, 163, 184);
      doc.setLineWidth(0.5);
    }
    doc.rect(marginX, photoY, photoSize, photoSize);
    textX = marginX + photoSize + 7;

  } catch (e) {
    console.error("Photo render failed:", e);
  }
}

doc.setFont("helvetica", "bold");
doc.setFontSize(isAi ? 17 : 16);
doc.setTextColor(isAi ? 255 : 15, isAi ? 255 : 23,
    isAi ? 255 : 42);
doc.text(previewData.name || "Your Name", textX, isAi
    ? 15 : 13, {
  maxWidth: pageWidth - textX - marginX,
});

doc.setFont("helvetica", "normal");
```

```
  doc.setFontSize(11.5);
  doc.text(previewData.headline || "Professional
      Headline", textX, isAi ? 22 : 20, {
    maxWidth: pageWidth - textX - marginX,
  });

  doc.setFontSize(10.5);
  const contactLine = `${previewData.email ? `Email:
      ${previewData.email}` : ""}${previewData.email &&
      previewData.phone ? "  |  " :
      ""}${previewData.phone ? `Phone:
      ${previewData.phone}` : ""}`;
  doc.text(contactLine || "Email: email@example.com  |
      Phone: +91 00000 00000", textX, isAi ? 29 : 26, {
    maxWidth: pageWidth - textX - marginX,
  });

  yPos = headerHeight + 10;
};

const drawSectionTitle = (title: string) => {
  ensureSpace(10);
  doc.setFont("helvetica", "bold");
  doc.setFontSize(12.5);
  doc.setTextColor(15, 23, 42);
  doc.text(title, marginX, yPos);
  doc.setDrawColor(accentR, accentG, accentB);
  doc.setLineWidth(0.7);
  doc.line(marginX, yPos + 1.8, pageWidth - marginX,
      yPos + 1.8);
  yPos += 7;
};

const drawParagraph = (text: string, color: [number,
    number, number] = [51, 65, 85]) => {
  if (!text.trim()) return;
  doc.setFont("helvetica", "normal");
  doc.setFontSize(10.8);
  doc.setTextColor(color[0], color[1], color[2]);
  const lines = doc.splitTextToSize(text, contentWidth);

  lines.forEach((line: string) => {
    ensureSpace(6);
    doc.text(line, marginX, yPos);
    yPos += 5.3;
  });
};

const drawBullets = (items: string[]) => {
```

```
  if (!items.length) return;
  doc.setFont("helvetica", "normal");
  doc.setFontSize(10.5);
  doc.setTextColor(51, 65, 85);
  items.forEach((item) => {
    const clean = item.replace(/^[-•\s]+/, "").trim();
    if (!clean) return;
    const wrapped = doc.splitTextToSize(clean,
        contentWidth - 6);
    wrapped.forEach((line: string, idx: number) => {
      ensureSpace(6);
      if (idx === 0) {
        doc.text(`- ${line}`, marginX, yPos);
      } else {
        doc.text(line, marginX + 4, yPos);
      }
      yPos += 5;
    });
  });
};

const drawSkills = (skillsList: string[]) => {
  if (!skillsList.length) return;
  if (!isAi) {
    drawParagraph(skillsList.join(" | "));
    return;
  }

  doc.setFont("helvetica", "normal");
  doc.setFontSize(9.8);
  let xPos = marginX;
  const rowHeight = 8;
  ensureSpace(rowHeight + 2);

  skillsList.forEach((skill) => {
    const text = skill.trim();
    if (!text) return;
    const chipW = Math.min(doc.getTextWidth(text) + 7,
        contentWidth);

    if (xPos + chipW > pageWidth - marginX) {
      xPos = marginX;
      yPos += rowHeight;
      ensureSpace(rowHeight + 2);
    }
    doc.setFillColor(accentR, accentG, accentB);
    doc.roundedRect(xPos, yPos - 4.8, chipW, 6.2, 2, 2,
        "F");
    doc.setTextColor(255, 255, 255);
```

```
    doc.text(text, xPos + 3.4, yPos - 0.6);
    xPos += chipW + 2.5;
  });
  yPos += rowHeight - 2;
};

try {
  drawHeader();

  if (previewData.summary) {
    drawSectionTitle("Professional Summary");
    drawParagraph(previewData.summary);
    yPos += 2;
  }

  const sectionEnabled = (resumeData as
      any)?.sectionEnabled || {
    education: true,
    projects: true,
    skills: true,
    languages: true,
    achievements: true,
    experience: true,
    certs: true,
  };

  const orderedSections = ((resumeData as
      any)?.order?.length
    ? (resumeData as any).order
    : ["education", "projects", "skills", "languages",
        "achievements", "experience", "certs"]
  ).filter((id: string) => ["education", "projects",
      "skills", "languages", "achievements",
      "experience", "certs"].includes(id));

  orderedSections.forEach((section: string) => {
    if (!sectionEnabled[section]) return;

    if (section === "skills") {
      const skillsList = previewData.skills
        .split(/[\n,]/)
        .map((s) => s.trim())

        .filter(Boolean);
      if (!skillsList.length) return;
      drawSectionTitle("Technical Skills");
      drawSkills(skillsList);
      yPos += 2;
      return;
```

```
}

if (section === "languages") {
  const list = (previewData.languages || "")
    .split(/[\n,]/)
    .map((x) => x.trim())
    .filter(Boolean);
  if (!list.length) return;
  drawSectionTitle("Languages");
  drawParagraph(list.join(" | "));
  yPos += 1;
  return;
}

if (section === "achievements") {
  const list = (previewData.achievements || "")
    .split("\n")
    .map((x) => x.trim())
    .filter(Boolean);
  if (!list.length) return;
  drawSectionTitle("Achievements");
  drawBullets(list);
  yPos += 1;
  return;
}

if (section === "education") {
  const list = previewData.education.filter((e) =>
      e.school || e.degree || e.year);
  if (!list.length) return;
  drawSectionTitle("Education");
  list.forEach((edu) => {
    ensureSpace(12);
    doc.setFont("helvetica", "bold");
    doc.setFontSize(11);
    doc.setTextColor(15, 23, 42);
    doc.text(edu.school || "Institution Name",
        marginX, yPos);
    yPos += 4.8;
    doc.setFont("helvetica", "normal");
    doc.setFontSize(10.4);

    doc.setTextColor(71, 85, 105);
    doc.text([edu.degree,
        edu.year].filter(Boolean).join(" | "),
        marginX, yPos);
    yPos += 5.8;
  });
  yPos += 1;
```

```
  return;
}

if (section === "projects") {
  const list = previewData.projects.filter((p) =>
      p.name || p.bullets);
  if (!list.length) return;
  drawSectionTitle("Projects");
  list.forEach((project) => {
    ensureSpace(10);
    doc.setFont("helvetica", "bold");
    doc.setFontSize(11);
    doc.setTextColor(15, 23, 42);
    doc.text(project.name || "Project", marginX,
        yPos);
    yPos += 5;
    const bullets = project.bullets
      .split("\n")
      .map((b) => b.trim())
      .filter(Boolean);
    drawBullets(bullets);
    yPos += 1.5;
  });
  return;
}

if (section === "experience") {
  const list = previewData.experience.filter((e) =>
      e.company || e.role || e.bullets);
  if (!list.length) return;
  drawSectionTitle("Experience");
  list.forEach((exp) => {
    ensureSpace(10);
    doc.setFont("helvetica", "bold");
    doc.setFontSize(11);
    doc.setTextColor(15, 23, 42);
    doc.text([exp.role,
        exp.company].filter(Boolean).join(" | ") ||
        "Role | Company", marginX, yPos);
    yPos += 5;
    const bullets = exp.bullets
      .split("\n")
      .map((b) => b.trim())
      .filter(Boolean);
    drawBullets(bullets);

    yPos += 1.5;
  });
  return;
```

```
  }

  if (section === "certs") {
    const list = previewData.certs.filter((c) =>
        c.name || c.org || c.year);
    if (!list.length) return;
    drawSectionTitle("Certifications");
    list.forEach((cert) => {
      ensureSpace(9);
      doc.setFont("helvetica", "bold");
      doc.setFontSize(10.8);
      doc.setTextColor(15, 23, 42);
      doc.text(cert.name || "Certification", marginX,
          yPos);
      yPos += 4.8;
      doc.setFont("helvetica", "normal");
      doc.setFontSize(10.3);
      doc.setTextColor(71, 85, 105);
      doc.text([cert.org,
          cert.year].filter(Boolean).join(" | "),
          marginX, yPos);
      yPos += 5.8;
    });
  }
});

if (isAi) {
  const bulletPool = [
    ...previewData.experience.flatMap((e) =>
        e.bullets.split("\n")),
    ...previewData.projects.flatMap((p) =>
        p.bullets.split("\n")),
  ]
    .map((line) => line.trim())
    .filter(Boolean)
    .slice(0, 3);

  if (bulletPool.length) {
    drawSectionTitle("Key Achievements");
    drawBullets(bulletPool);
  }
}

const totalPages = (doc as
    any).internal.getNumberOfPages ? (doc as
    any).internal.getNumberOfPages() :
    doc.getNumberOfPages();
for (let page = 1; page <= totalPages; page += 1) {
```

```
      doc.setPage(page);
      doc.setFont("helvetica", "normal");
      doc.setFontSize(9);

      doc.setTextColor(100, 116, 139);
      doc.text(`${previewData.name || "Resume"}  •  Page
          ${page} of ${totalPages}`, pageWidth / 2,
          pageHeight - 7, {
        align: "center",
      });
    }

    const fileName = `${(previewData.name ||
        "resume").replace(/\s+/g, "_")}_resume.pdf`;
    doc.save(fileName);
  } catch (error: any) {
    console.error("PDF Generation failed:", error);
  }
};

if (!loaded) {
  return <div className="p-8 text-
      center">Loading...</div>;
}

return (
  <div className="mx-auto w-full max-w-6xl">
    <div className="mb-6">
      <Button variant="ghost" asChild>
        <Link to="/create">
          <ArrowLeft className="mr-2 h-4 w-4" />
          Back to Resume Builder
        </Link>
      </Button>
    </div>

    <h1 className="text-2xl font-semibold tracking-tight
        text-foreground">Export Resume</h1>
    <p className="mt-1 text-muted-foreground">
      Template: {template?.name} - Choose Manual or AI
          Enhanced format.
    </p>

    <div className="mt-6 grid gap-6 lg:grid-cols-2">

      <Card className={`${choice === "manual" ? "ring-2
          ring-primary" : ""} border-2 border-slate-200
          bg-white text-slate-900 dark:bg-white dark:text-
          slate-900`}>
```

```
<CardHeader className="border-b border-slate-200
    bg-slate-50 flex flex-row items-center
    justify-between">
  <div>
    <CardTitle className="flex items-center gap-2
        text-black">
      <span className="text-2xl">■</span> Manual
          Resume
    </CardTitle>
    <CardDescription className="text-gray-600
        mt-1">Your original unedited
        content.</CardDescription>
  </div>
  <div className="flex flex-col items-center
      justify-center h-16 w-16 rounded-full
      border-4 border-amber-400 bg-amber-50">
    <span className="text-lg font-bold text-
        amber-600">62%</span>

    <span className="text-[9px] uppercase font-
        bold text-amber-600 tracking-tighter">ATS
        Score</span>
  </div>
</CardHeader>
<CardContent>
  <Button
    variant={choice === "manual" ? "default" :
        "outline"}
    onClick={() => setChoice("manual")}
    className="mb-4 w-full"
  >
    Select Manual
  </Button>

  {/* Manual Preview - VERY Simple */}
  <div className="rounded-lg border border-
      gray-300 bg-white p-4 text-black shadow-sm">
    {/* Plain Header */}
    <div className="border-b-2 border-black pb-2
        mb-3 flex items-center gap-3">
      {previewData.photoDataUrl && (
        <img
          src={previewData.photoDataUrl}
          alt="Profile"
          className="h-12 w-12 rounded-full border
              border-gray-400 object-cover flex-
              shrink-0"
        />
      )}
```

```
  <div className={previewData.photoDataUrl ?
      "" : "w-full text-center"}>
    <h2 className="text-lg font-bold text-
        black">{previewData.name}</h2>
    <p className="text-sm text-
        black">{previewData.headline}</p>
    <p className="text-xs text-gray-700
        mt-1">■ {previewData.email}
        {previewData.phone ? `| ■
        ${previewData.phone}` : ""}</p>
  </div>
</div>

{/* Simple Summary */}
{previewData.summary && (
  <div className="mb-3">
    <h3 className="font-bold text-sm text-
        black border-b border-gray-400
        mb-1">Summary</h3>
    <p className="text-xs text-
        gray-900">{previewData.summary}</p>
  </div>
)}

{/* Simple Skills */}
{previewData.skills && (
  <div className="mb-3">
    <h3 className="font-bold text-sm text-
        black border-b border-gray-400
        mb-1">Skills</h3>
    <p className="text-xs text-
        gray-900">{previewData.skills}</p>
  </div>
)}

{/* Simple Education */}
{previewData.education &&
    previewData.education.length > 0 && (
  <div>
    <h3 className="font-bold text-sm text-
        black border-b border-gray-400
        mb-1">Education</h3>
    {previewData.education.map((edu, i) => (
      <p key={i} className="text-xs text-
          gray-600">{edu.school} -
          {edu.degree} ({edu.year})</p>
    ))}
  </div>
)}
```

```
    </div>
  </CardContent>
</Card>

<Card className={`${choice === "ai" ? "ring-2 ring-
    primary" : ""} border-2 border-slate-200 bg-
    white text-slate-900 dark:bg-white dark:text-
    slate-900`}>
  <CardHeader className="border-b border-slate-200
      bg-emerald-50/50 flex flex-row items-center
      justify-between">
    <div>
      <CardTitle className="flex items-center gap-2
          text-black">
        <span className="text-2xl">■</span> AI
            Enhanced Resume
      </CardTitle>
      <CardDescription className="text-gray-600
          mt-1">Opus 4.6 improved with ATS
          keywords.</CardDescription>
    </div>
    <div className="flex flex-col items-center
        justify-center h-16 w-16 rounded-full
        border-4 border-emerald-500 bg-emerald-50">
      <span className="text-lg font-bold text-
          emerald-600">95%</span>
      <span className="text-[9px] uppercase font-
          bold text-emerald-600 tracking-
          tighter">ATS Score</span>
    </div>
  </CardHeader>
  <CardContent>
    <Button
      variant={choice === "ai" ? "default" :
          "outline"}
      onClick={() => setChoice("ai")}
      className="mb-4 w-full bg-gradient-to-r from-
          primary to-secondary text-white
          hover:opacity-95"
    >
      ■ Select AI Enhanced
    </Button>

    {/* AI Enhanced Preview - Beautiful Professional
        */}
    <div
      className="rounded-lg border-2 shadow-xl"
      style={{
```

```
        background: "linear-gradient(180deg, #ffffff
            0%, #f7f9fc 100%)",
        borderColor: "#2563eb"
      }}
    >
      {/* Beautiful Header with Gradient */}

      <div
        className="rounded-t-lg px-4 py-4 text-white
            flex items-center gap-3"
        style={{
          background: "linear-gradient(135deg,
              #2563eb 0%, #0891b2 50%, #2563eb
              100%)",
          boxShadow: "0 4px 15px rgba(37, 99, 235,
              0.35)"
        }}
      >
        {previewData.photoDataUrl && (
          <img
            src={previewData.photoDataUrl}
            alt="Profile"
            className="h-14 w-14 rounded-full
                border-2 border-white object-cover
                flex-shrink-0"
          />
        )}
        <div className={previewData.photoDataUrl ?
            "" : "w-full text-center"}>
```

Result Output:

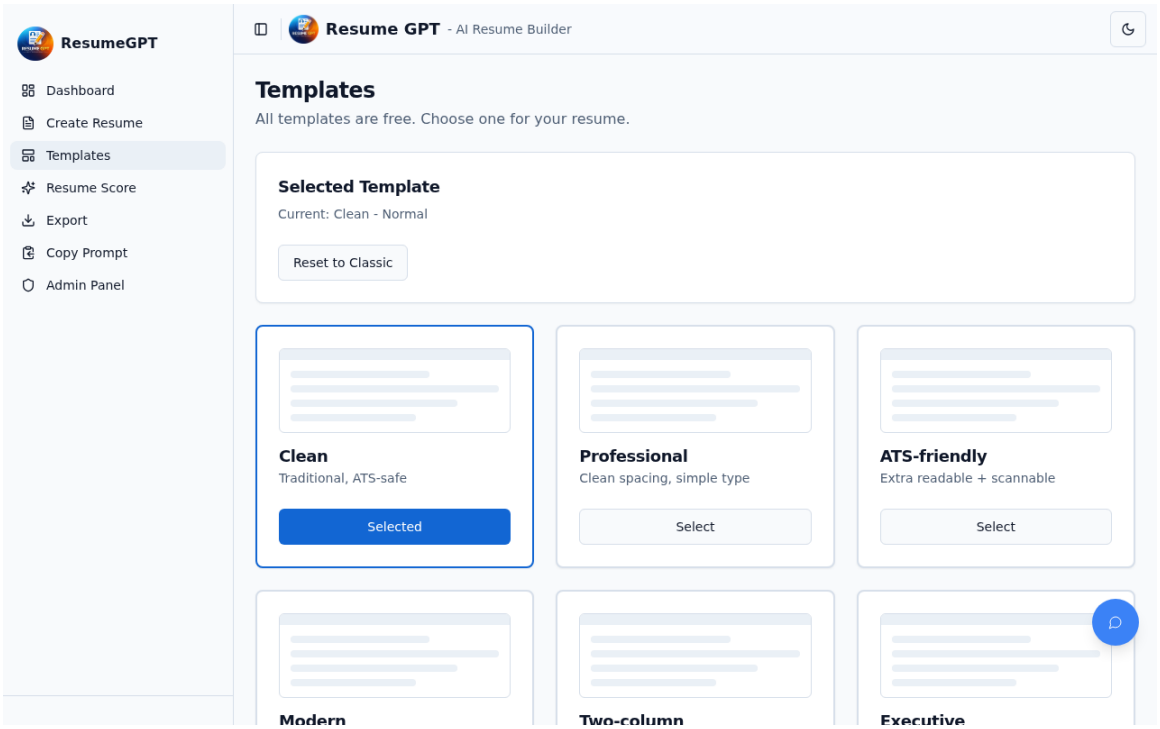

Figure 6: Practical System Output

Practical No. 7

Aim: To implement a custom React hook for debounced data persistence.

Source Code Implementation:

```
import { useState, useEffect, useCallback, useRef } from
    "react";
import { useResumes, Resume } from "@/hooks/useResumes";

const AUTOSAVE_INTERVAL = 10_000; // 10 seconds

export function useAutoSave(
  resumeId: string,
  currentFormState: any,
  selectedTemplate: string,
  sectionOrder: string[],
  sectionEnabled: Record<string, boolean>
) {
  const { updateResume } = useResumes();
  const [isSaving, setIsSaving] = useState(false);
  const [lastSavedAt, setLastSavedAt] = useState<Date |
      null>(null);
  const [hasUnsavedChanges, setHasUnsavedChanges] =
      useState(false);

  const lastSavedState = useRef({
    data: JSON.stringify(currentFormState),
    template_id: selectedTemplate,
    section_order: JSON.stringify(sectionOrder),
    section_enabled: JSON.stringify(sectionEnabled)
  });

  // Track if changes occurred compared to last saved state
  useEffect(() => {
    const currentStateStr = {
      data: JSON.stringify(currentFormState),
      template_id: selectedTemplate,
      section_order: JSON.stringify(sectionOrder),
      section_enabled: JSON.stringify(sectionEnabled)
    };

    if (
      currentStateStr.data !== lastSavedState.current.data
          ||
      currentStateStr.template_id !==
          lastSavedState.current.template_id ||
      currentStateStr.section_order !==
          lastSavedState.current.section_order ||
```

```
    currentStateStr.section_enabled !==
        lastSavedState.current.section_enabled
  ) {
    setHasUnsavedChanges(true);
  }
}, [currentFormState, selectedTemplate, sectionOrder,
    sectionEnabled]);

const forceSave = useCallback(async () => {
  if (!resumeId || !hasUnsavedChanges) return;

  setIsSaving(true);
  const success = await updateResume(resumeId, {
    data: currentFormState,
    template_id: selectedTemplate,
    section_order: sectionOrder,
    section_enabled: sectionEnabled
  });

  if (success) {
    setHasUnsavedChanges(false);
    setLastSavedAt(new Date());
    // Update last saved ref
    lastSavedState.current = {
      data: JSON.stringify(currentFormState),
      template_id: selectedTemplate,
      section_order: JSON.stringify(sectionOrder),
      section_enabled: JSON.stringify(sectionEnabled)
    };
  }
  setIsSaving(false);
}, [resumeId, currentFormState, selectedTemplate,
    sectionOrder, sectionEnabled, hasUnsavedChanges,
    updateResume]);

// Auto-save timer
useEffect(() => {
  const timer = setInterval(() => {
    forceSave();
  }, AUTOSAVE_INTERVAL);

  return () => clearInterval(timer);
}, [forceSave]);

// Save before unload (closing tab)
useEffect(() => {
  const handleBeforeUnload = (e: BeforeUnloadEvent) => {
    if (hasUnsavedChanges) {
      e.preventDefault();
```

```
            e.returnValue = ''; // Required for Chrome to show
                prompt
          }
        };
        window.addEventListener('beforeunload',
            handleBeforeUnload);
        return () => window.removeEventListener('beforeunload',
            handleBeforeUnload);
      }, [hasUnsavedChanges]);

      return { isSaving, lastSavedAt, hasUnsavedChanges,
          forceSave };

    }
```

Result Output:

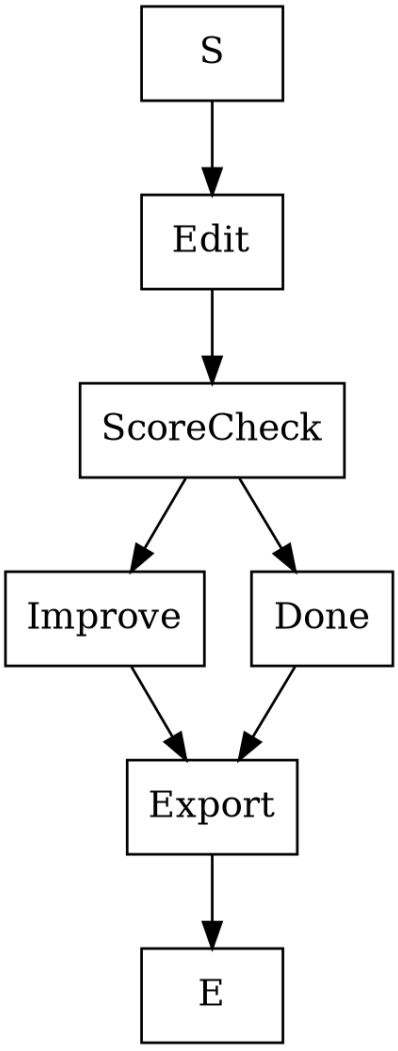

Figure 7: Practical System Output

Practical No. 8

Aim: To create an administrative control panel for monitoring platform-wide metrics.

Source Code Implementation:

```
import { useEffect, useMemo, useState } from "react";
import { BarChart3, Check, Eye, EyeOff, KeyRound,
    RefreshCcw, Shield, Trash2 } from "lucide-react";

import { Badge } from "@/components/ui/badge";
import { Button } from "@/components/ui/button";
import { Card, CardContent, CardDescription, CardHeader,
    CardTitle } from "@/components/ui/card";
import { Input } from "@/components/ui/input";
import { Tabs, TabsContent, TabsList, TabsTrigger } from
    "@/components/ui/tabs";
import { useToast } from "@/hooks/use-toast";
import {
  TEMPLATE_IDS,
  getActiveApiKeyId,
  getMetricsSnapshot,
  getStoredApiKeys,
  getTemplateVisibility,
  resetDemoData,
  setActiveApiKeyId,
  setStoredApiKeys,
  setTemplateVisibility,
  type StoredApiKey,
} from "@/lib/demoStorage";

const ADMIN_USER = "vansh123";
const ADMIN_PASS = "philip99";

const labelFromTemplateId = (id: string) =>
  id
    .replace(/([a-z])([A-Z])/g, "$1 $2")
    .replace(/^\w/, (c) => c.toUpperCase())
    .replace("Twocol", "Two-column")
    .replace("Atspro", "ATS-friendly");

export default function Admin() {
  const { toast } = useToast();

  const [isLoggedIn, setIsLoggedIn] = useState(false);
  const [username, setUsername] = useState("");
  const [password, setPassword] = useState("");
  const [showPassword, setShowPassword] = useState(false);
  const [error, setError] = useState("");
```

```
const [keys, setKeys] = useState<StoredApiKey[]>(() =>
    getStoredApiKeys());
const [activeKeyId, setActiveKeyIdState] = useState(() =>
    getActiveApiKeyId());
const [newKeyLabel, setNewKeyLabel] = useState("");
const [newKeyValue, setNewKeyValue] = useState("");

const [visibility, setVisibility] =
    useState<Record<string, boolean>>(() =>
    getTemplateVisibility());
const [metricsRefresh, setMetricsRefresh] = useState(0);

const metrics = useMemo(() => getMetricsSnapshot(),
    [metricsRefresh]);

useEffect(() => {
  localStorage.removeItem("admin_logged_in");
}, []);

const saveKeys = (next: StoredApiKey[]) => {
  setKeys(next);
  setStoredApiKeys(next);
};

const setActiveKey = (id: string) => {
  setActiveKeyIdState(id);
  setActiveApiKeyId(id);
  toast({ title: "Active key switched", description: "AI
      features will use this key." });
};

const addKey = () => {
  const value = newKeyValue.trim();
  if (!value) {
    toast({ variant: "destructive", title: "API key
        required" });
    return;
  }
  const item: StoredApiKey = {
    id: `key_${Date.now()}`,
    label: newKeyLabel.trim() || `Key ${keys.length + 1}`,
    key: value,
  };
  const next = [item, ...keys];
  saveKeys(next);
  if (!activeKeyId) setActiveKey(item.id);
  setNewKeyLabel("");
  setNewKeyValue("");
  toast({ title: "API key added", description: "Stored
```

```
      locally for demo." });
};

const removeKey = (id: string) => {
  const next = keys.filter((k) => k.id !== id);
  saveKeys(next);
  if (activeKeyId === id) {
    const fallback = next[0]?.id || "";
    setActiveKeyIdState(fallback);

    setActiveApiKeyId(fallback);
  }
};

const toggleTemplate = (id: string) => {
  const next = { ...visibility, [id]: !(visibility[id] ??
      true) };
  setVisibility(next);
  setTemplateVisibility(next);
};

const handleLogin = () => {
  if (username === ADMIN_USER && password === ADMIN_PASS)
      {
    localStorage.setItem("admin_logged_in", "true");
    setIsLoggedIn(true);
    setError("");
    toast({ title: "Admin login successful" });
    return;
  }
  setError("Invalid username or password");
};

const logout = () => {
  localStorage.removeItem("admin_logged_in");
  setIsLoggedIn(false);
  setUsername("");
  setPassword("");
};

if (!isLoggedIn) {
  return (
    <div className="min-h-screen bg-background p-4">
      <div className="mx-auto flex min-h-[80vh] w-full
          max-w-md items-center">
        <Card className="w-full">
          <CardHeader className="text-center">
            <div className="mx-auto mb-3 flex h-14 w-14
                items-center justify-center rounded-full
```

```
                bg-primary">
              <Shield className="h-7 w-7 text-primary-
                  foreground" />
            </div>
            <CardTitle>Admin Login</CardTitle>
            <CardDescription>Use credentials to manage AI
                keys and demo settings.</CardDescription>
          </CardHeader>
          <CardContent className="space-y-3">
            <Input placeholder="Username" value={username}
                onChange={(e) =>
                setUsername(e.target.value)} />
            <div className="relative">
              <Input
                type={showPassword ? "text" : "password"}

                placeholder="Password"
                value={password}
                onChange={(e) =>
                    setPassword(e.target.value)}
                className="pr-10"
              />
              <button
                type="button"
                onClick={() => setShowPassword((prev) =>
                    !prev)}
                className="absolute right-3 top-1/2
                    -translate-y-1/2 text-muted-foreground
                    hover:text-foreground"
                aria-label={showPassword ? "Hide password"
                    : "Show password"}
              >
                {showPassword ? <EyeOff className="h-4
                    w-4" /> : <Eye className="h-4 w-4" />}
              </button>
            </div>
            {error ? <p className="text-sm text-
                red-500">{error}</p> : null}
            <Button className="w-full"
                onClick={handleLogin}>
              Login
            </Button>
          </CardContent>
        </Card>
      </div>
    </div>
  );
}
```

```
return (
  <div className="min-h-screen bg-background p-6">
    <div className="mx-auto w-full max-w-6xl">
      <div className="mb-6 flex flex-wrap items-end
          justify-between gap-3">
        <div>
          <h1 className="flex items-center gap-2 text-2xl
              font-semibold">
            <Shield className="h-6 w-6 text-primary" />
            Admin Panel
          </h1>
          <p className="text-muted-foreground">Manage API
              keys, templates, and demo analytics.</p>
        </div>
        <Button variant="outline" onClick={logout}>
          Logout
        </Button>
      </div>

      <Tabs defaultValue="keys">
        <TabsList className="grid w-full grid-cols-3">
          <TabsTrigger value="keys">AI Keys</TabsTrigger>
          <TabsTrigger value="templates">Template
              Visibility</TabsTrigger>

          <TabsTrigger value="analytics">Analytics &
              Reset</TabsTrigger>
        </TabsList>

        <TabsContent value="keys" className="mt-6
            space-y-6">
          <Card>
            <CardHeader>
              <CardTitle className="flex items-center
                  gap-2">
                <KeyRound className="h-5 w-5" />
                Add AI API Key
              </CardTitle>
              <CardDescription>Multiple keys supported.
                  Active key is used by all AI
                  features.</CardDescription>
            </CardHeader>
            <CardContent className="grid gap-3">
              <Input
                placeholder="Label (e.g. Team Key 1)"
                value={newKeyLabel}
                onChange={(e) =>
                    setNewKeyLabel(e.target.value)}
              />
```

```
    <Input
      placeholder="Paste API key"
      value={newKeyValue}
      onChange={(e) =>
          setNewKeyValue(e.target.value)}
    />
    <Button onClick={addKey}>Add Key</Button>
  </CardContent>
</Card>

<Card>
  <CardHeader>
    <CardTitle>Stored Keys</CardTitle>
    <CardDescription>Switch active key or remove
        old keys.</CardDescription>
  </CardHeader>
  <CardContent className="space-y-2">
    {keys.length === 0 ? (
      <p className="text-sm text-muted-
          foreground">No keys saved yet.</p>
    ) : (
      keys.map((item) => {
        const active = item.id === activeKeyId;
        return (
          <div key={item.id} className="flex
              items-center justify-between
              rounded-lg border p-3">
            <div className="min-w-0">
              <p className="truncate font-
                  medium">{item.label}</p>
              <p className="truncate text-xs
                  text-muted-
                  foreground">{item.key.slice(0,
                  14)}...</p>
            </div>
            <div className="flex items-center
                gap-2">

              {active ? <Badge>Active</Badge> :
                  null}
              <Button size="sm" variant={active
                  ? "secondary" : "outline"}
                  onClick={() =>
                  setActiveKey(item.id)}>
                {active ? <Check className="h-4
                    w-4" /> : "Activate"}
              </Button>
              <Button size="sm"
                  variant="outline" onClick={()
```

```
                    => removeKey(item.id)}>
                  <Trash2 className="h-4 w-4" />
                </Button>
              </div>
            </div>
          );
        })
      )}
    </CardContent>
  </Card>
</TabsContent>

<TabsContent value="templates" className="mt-6">
  <Card>
    <CardHeader>
      <CardTitle>Template Visibility
          Control</CardTitle>
      <CardDescription>Enable or hide templates
          for demo users.</CardDescription>
    </CardHeader>
    <CardContent className="grid gap-2 sm:grid-
        cols-2 lg:grid-cols-3">
      {TEMPLATE_IDS.map((id) => {
        const enabled = visibility[id] ?? true;
        return (
          <button
            key={id}
            type="button"
            onClick={() => toggleTemplate(id)}
            className={`rounded-lg border px-4
                py-3 text-left transition
                ${enabled ? "border-emerald-300
                bg-emerald-50 dark:bg-
                emerald-950/20" : "border-
                slate-300 bg-slate-100 dark:bg-
                slate-900/40"}`}
          >
            <p className="font-
                medium">{labelFromTemplateId(id)}<
                /p>
            <p className="text-xs text-muted-
                foreground">{enabled ? "Visible to
                users" : "Hidden from users"}</p>
          </button>
        );
      })}
    </CardContent>
  </Card>
```

```
</TabsContent>

<TabsContent value="analytics" className="mt-6
    space-y-6">
  <Card>
    <CardHeader>
      <CardTitle className="flex items-center
          gap-2">

        <BarChart3 className="h-5 w-5" />
        Demo Counters
      </CardTitle>
      <CardDescription>Locally tracked usage
          counters.</CardDescription>
    </CardHeader>
    <CardContent className="grid gap-3 sm:grid-
        cols-3">
      <div className="rounded-lg border p-4">
        <p className="text-sm text-muted-
            foreground">Resumes created</p>
        <p className="mt-1 text-2xl font-
            semibold">{metrics.resumesCreated}</p>
      </div>
      <div className="rounded-lg border p-4">
        <p className="text-sm text-muted-
            foreground">AI usage count</p>
        <p className="mt-1 text-2xl font-
            semibold">{metrics.aiUsage}</p>
      </div>
      <div className="rounded-lg border p-4">
        <p className="text-sm text-muted-
            foreground">Templates used</p>
        <p className="mt-1 text-2xl font-
            semibold">{Object.values(metrics.templ
            ateUsage).reduce((a, b) => a + b,
            0)}</p>
      </div>
    </CardContent>
  </Card>

  <Card>
    <CardHeader>
      <CardTitle>Reset Demo Data</CardTitle>
      <CardDescription>Clears local resume data
          and usage counters.</CardDescription>
    </CardHeader>
    <CardContent className="flex flex-wrap gap-3">
      <Button variant="outline" onClick={() =>
          setMetricsRefresh((x) => x + 1)}>
```

```
                    <RefreshCcw className="mr-2 h-4 w-4" />
                    Refresh Counters
                  </Button>
                  <Button
                    variant="destructive"
                    onClick={() => {
                      resetDemoData();
                      setMetricsRefresh((x) => x + 1);
                      toast({ title: "Demo data reset
                          complete" });
                    }}
                  >
                    <Trash2 className="mr-2 h-4 w-4" />
                    Reset Demo Data
                  </Button>
                </CardContent>
              </Card>
            </TabsContent>

          </Tabs>
        </div>
      </div>
    );
  }
```

Result Output:

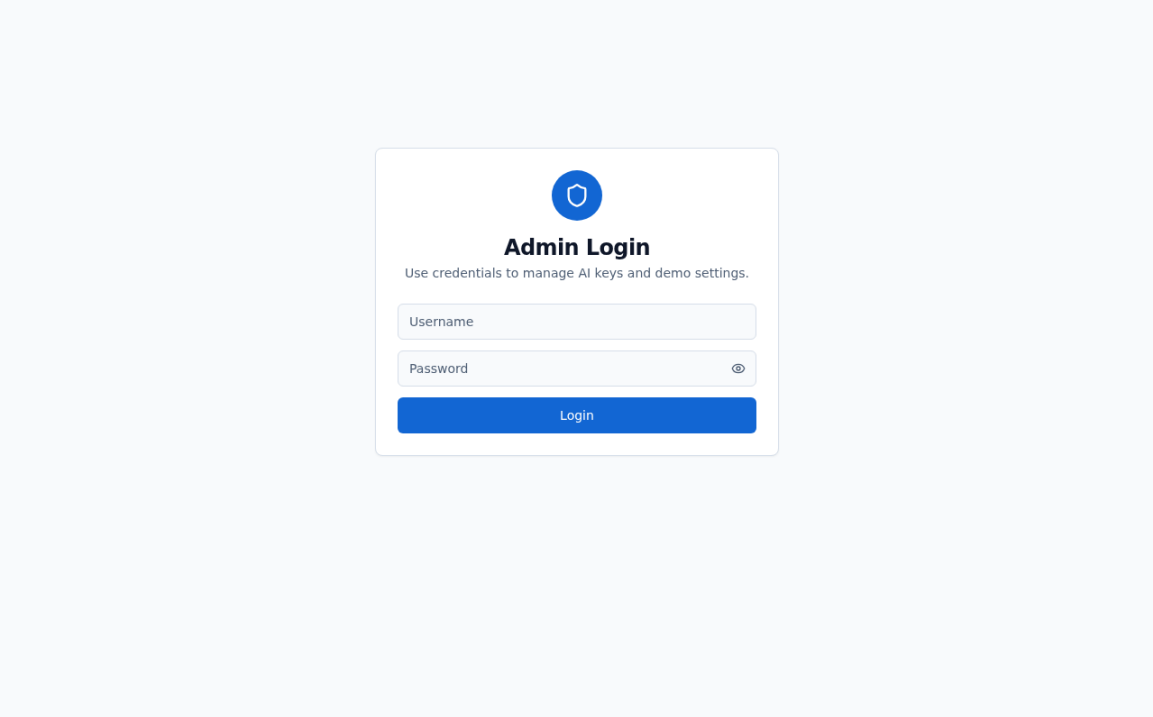

Figure 8: Practical System Output

Practical No. 9

Aim: To implement secure JWT-based authentication using Supabase Auth.

Source Code Implementation:

```
import { useState } from "react";
import { Card, CardContent, CardDescription, CardHeader,
    CardTitle, CardFooter } from "@/components/ui/card";
import { Button } from "@/components/ui/button";
import { Input } from "@/components/ui/input";
import { Label } from "@/components/ui/label";
import { useToast } from "@/hooks/use-toast";
import { supabase } from "@/integrations/supabase/client";
import { useNavigate, useLocation } from "react-router-dom";

export default function Auth() {
  const [isLogin, setIsLogin] = useState(true);
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [loading, setLoading] = useState(false);
  const { toast } = useToast();
  const navigate = useNavigate();
  const location = useLocation();

  const from = location.state?.from?.pathname ||
      "/dashboard";

  const handleAuth = async (e: React.FormEvent) => {
    e.preventDefault();
    setLoading(true);

    try {
      if (isLogin) {
        const { error } = await
            supabase.auth.signInWithPassword({
          email,
          password,
        });
        if (error) throw error;
        toast({ title: "Welcome back!" });
        navigate(from, { replace: true });
      } else {
        const { error } = await supabase.auth.signUp({
          email,
          password,
          options: {
            data: {
              full_name: email.split("@")[0],
```

```
          }
        }
      });
      if (error) throw error;
      toast({ title: "Registration successful" });

      navigate(from, { replace: true });
    }
  } catch (error: any) {
    toast({ variant: "destructive", title: "Error",
        description: error.message });
  } finally {
    setLoading(false);
  }
};

return (
  <div className="flex min-h-screen items-center justify-
      center py-20 px-4 bg-muted/30">
    <Card className="w-full max-w-md mx-auto">
      <CardHeader className="text-center">
        <CardTitle>{isLogin ? "Welcome back" : "Create an
            account"}</CardTitle>
        <CardDescription>
          {isLogin
            ? "Enter your credentials to access your
                account"
            : "Sign up to start building your professional
                resume"}
        </CardDescription>
      </CardHeader>
      <CardContent>
        <form onSubmit={handleAuth} className="space-y-4">
          <div className="space-y-2">
            <Label htmlFor="email">Email</Label>
            <Input
              id="email"
              type="email"
              placeholder="you@example.com"
              value={email}
              onChange={(e) => setEmail(e.target.value)}
              required
            />
          </div>
          <div className="space-y-2">
            <Label htmlFor="password">Password</Label>
            <Input
              id="password"
              type="password"
```

```
                    value={password}
                    onChange={(e) =>
                        setPassword(e.target.value)}
                    required
                    minLength={6}
                  />
                </div>
                <Button className="w-full" type="submit"
                    disabled={loading}>

                  {loading ? "Please wait..." : (isLogin ? "Sign
                      In" : "Sign Up")}
                </Button>
              </form>
            </CardContent>
            <CardFooter className="flex justify-center">
              <Button variant="link" onClick={() =>
                  setIsLogin(!isLogin)} className="text-sm">
                {isLogin ? "Don't have an account? Sign up" :
                    "Already have an account? Sign in"}
              </Button>
            </CardFooter>
          </Card>
        </div>
      );
    }
```

Result Output:

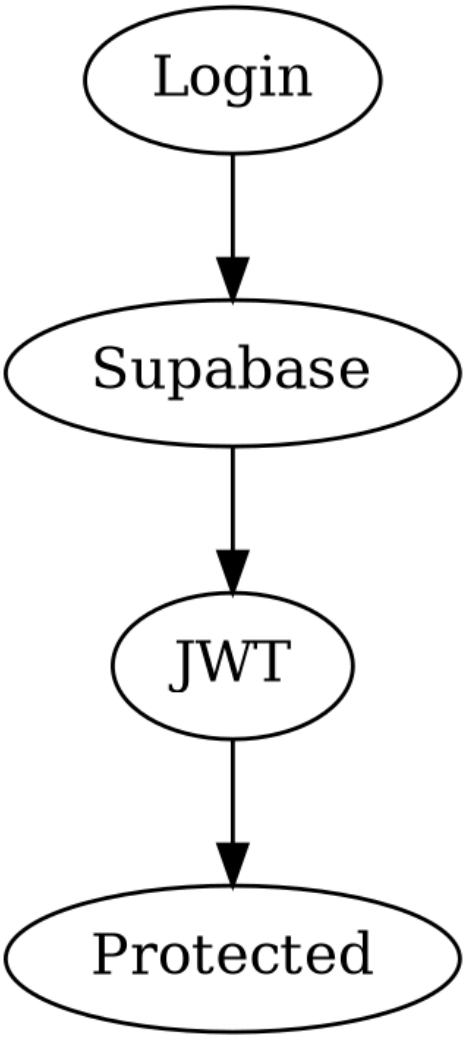

Figure 9: Practical System Output

Practical No. 10

Aim: To design a scalable project architecture with global routing.

Source Code Implementation:

```
import { Toaster } from "@/components/ui/toaster";
import { Toaster as Sonner } from "@/components/ui/sonner";
import { TooltipProvider } from "@/components/ui/tooltip";
import { QueryClient, QueryClientProvider } from
    "@tanstack/react-query";
import { BrowserRouter, Routes, Route, Navigate } from
    "react-router-dom";
import { ThemeProvider } from
    "@/components/theme/ThemeProvider";
import { DashboardLayout } from
    "./components/app/DashboardLayout";
import DashboardHome from "./pages/dashboard/DashboardHome";
import ResumeBuilder from "./pages/dashboard/ResumeBuilder";
import Templates from "./pages/dashboard/Templates";
import ResumeScore from "./pages/dashboard/ResumeScore";
import ExportResume from "./pages/dashboard/ExportResume";
import Admin from "./pages/Admin";
import NotFound from "./pages/NotFound";
import PromptPage from "./pages/Prompt";
import Index from "./pages/Index";
import Auth from "./pages/Auth";
import { AuthProvider } from "./contexts/AuthContext";
import { ProtectedRoute } from
    "./components/auth/ProtectedRoute";

const queryClient = new QueryClient();

const App = () => (
  <QueryClientProvider client={queryClient}>
    <ThemeProvider>
      <TooltipProvider>
        <Toaster />
        <Sonner />
        <BrowserRouter>
          <Routes>
            <Route path="/admin" element={<Admin />} />
            {/* Public landing page */}
            <Route path="/" element={<Index />} />
            <Route path="/auth" element={<Navigate
                to="/dashboard" replace />} />

            {/* Main app */}
            <Route element={<DashboardLayout />}>
```

```
                <Route path="/dashboard"
                    element={<DashboardHome />} />
                <Route path="/create" element={<ResumeBuilder
                    />} />
                <Route path="/create/:id"
                    element={<ResumeBuilder />} />
                <Route path="/templates" element={<Templates
                    />} />
                <Route path="/score/:id" element={<ResumeScore
                    />} />
                <Route path="/export" element={<ExportResume
                    />} />
                <Route path="/export/:id"
                    element={<ExportResume />} />
                <Route path="/prompt" element={<PromptPage />}
                    />

              </Route>

              {/* Backwards-compat */}
              <Route path="/dashboard/*" element={<Navigate
                  to="/dashboard" replace />} />
              <Route path="/score" element={<Navigate
                  to="/dashboard" replace />} />
              <Route path="/export" element={<Navigate
                  to="/dashboard" replace />} />
              {/* ADD ALL CUSTOM ROUTES ABOVE THE CATCH-ALL
                  "*" ROUTE */}
              <Route path="*" element={<NotFound />} />
            </Routes>
          </BrowserRouter>
        </TooltipProvider>
      </ThemeProvider>
    </QueryClientProvider>
  );

  export default App;
```

Result Output:

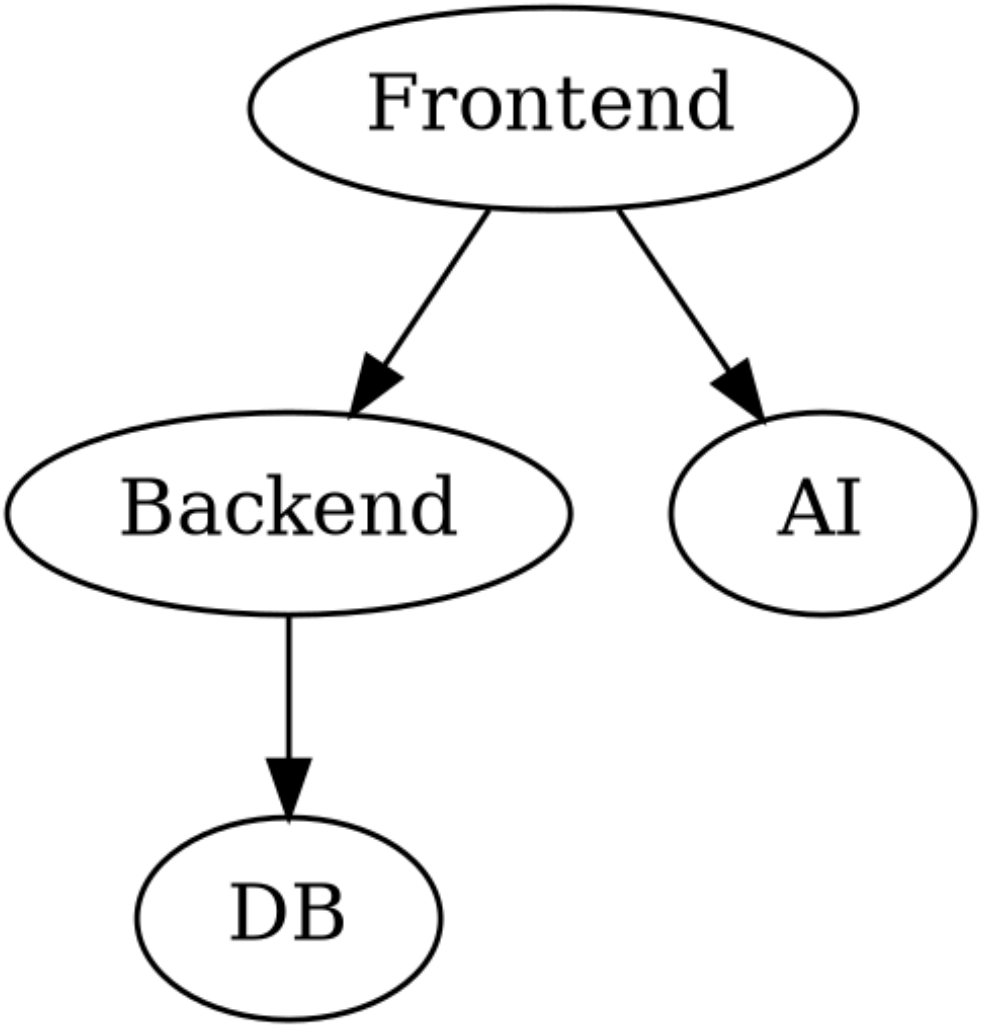

Figure 10: Practical System Output

Practical No. 11

Aim: To implement custom business logic hooks for resume management.

Source Code Implementation:

```
import { useState, useCallback, useEffect } from "react";
import { useToast } from "@/hooks/use-toast";

export interface Resume {
  id: string;
  title: string;
  data: any;
  template_id: string;
  section_order: string[];
  section_enabled: Record<string, boolean>;
  last_score: number | null;
  updated_at: string;
  is_archived: boolean;
}

export function useResumes() {
  const { toast } = useToast();
  const [resumes, setResumes] = useState<Resume[]>([]);
  const [loading, setLoading] = useState(false);

  // Load from local storage
  const fetchResumes = useCallback(async () => {
    setLoading(true);
    try {
      const stored = localStorage.getItem("LOCAL_RESUMES");
      if (stored) {
        const parsed: Resume[] = JSON.parse(stored);
        const active = parsed.filter(r =>
            !r.is_archived).sort((a,b) => new
            Date(b.updated_at).getTime() - new
            Date(a.updated_at).getTime());
        setResumes(active);
      } else {
        // Create an ultra-professional premium resume by
            default (Alex Doe - Senior Engineer)
        const premiumResume = {
          id: crypto.randomUUID(),
          title: "Senior Product Architect",
          template_id: "executive",
          section_order: ["experience", "education",
              "skills", "projects", "languages",
              "achievements", "certs"],
          section_enabled: { experience: true, education:
```

```
    true, skills: true, projects: true, languages:
    true, achievements: true, certs: true },
last_score: 95,
updated_at: new Date().toISOString(),
is_archived: false,
data: {
  name: "Alex Khandekar",
  headline: "Senior SaaS Product Architect",
  email: "alex.pro@example.com",
  phone: "+1 (555) 019-2831",

  photoDataUrl: "https://images.unsplash.com/photo
      -1560250097-
      0b93528c311a?auto=format&fit=crop&q=80&w=400
      ",
  summary: "Visionary Software Architect with 8+
      years of experience leading high-performance
      engineering teams. Expert in distributed
      systems, modern React ecosystems, and
      orchestrating massive cloud migrations.
      Proven track record of increasing system
      efficiency by 40% and mentoring 20+
      engineers into leadership roles.",
  experience: [
    { role: "VP of Engineering", company:
        "NexusTech Solutions", bullets:
        "Orchestrated the architectural migration
        of a legacy monolith to a globally
        distributed microservices architecture,
        reducing latency by 45%.\nLed a team of
        45+ engineers across 3 continents,
        delivering the flagship SaaS product 2
        months ahead of schedule.\nSpearheaded the
        integration of AI-driven analytics, which
        increased customer retention by 22%
        quarter-over-quarter." },
    { role: "Senior Backend Developer", company:
        "FinServe Analytics", bullets: "Designed
        and implemented a real-time transaction
        processing pipeline capable of handling
        50,000 TPS with 99.999% uptime.\nOptimized
        PostgreSQL database queries, reducing
        average query time from 400ms to under
        15ms.\nMentored junior developers and
        established CI/CD pipelines that cut
        deployment times in half." }
  ],
  education: [
    { school: "Stanford University", degree: "M.S.
```

```
                in Computer Science", year: "2016" },
            { school: "Indian Institute of Technology
                (IIT)", degree: "B.Tech in Software
                Engineering", year: "2014" }
          ],
          skills: "React, Node.js, TypeScript, PostgreSQL,
              AWS Cloud, Docker/Kubernetes, Architecture
              Design, Team Leadership, Agile/Scrum",
          projects: [],
          languages: "English (Native), Hindi (Fluent),
              Spanish (Conversational)",
          achievements: "Winner of the Silicon Valley
              Coding Hackathon 2018\nPublished 3 research
              papers on scalable cloud architectures",
          certs: []
        }
      };
      localStorage.setItem("LOCAL_RESUMES",
          JSON.stringify([premiumResume]));
      setResumes([premiumResume]);
    }
  } catch (e) {
    console.error(e);
  }
  setLoading(false);
}, []);

const saveToLocal = (newResumes: Resume[]) => {
  localStorage.setItem("LOCAL_RESUMES",
      JSON.stringify(newResumes));
};

const createResume = async (title: string = "Untitled
    Resume") => {
  const newResume: Resume = {
    id: crypto.randomUUID(),
    title,
    data: {},
    template_id: "classic",
    section_order: ["education", "projects", "skills",
        "languages", "achievements", "experience",
        "certs"],
    section_enabled: {},
    last_score: null,
    updated_at: new Date().toISOString(),
    is_archived: false
  };

  const stored = localStorage.getItem("LOCAL_RESUMES");
```

```
  const parsed: Resume[] = stored ? JSON.parse(stored) :
      [];

  const updated = [newResume, ...parsed];
  saveToLocal(updated);

  setResumes(prev => [newResume, ...prev]);
  return newResume;
};

const getResume = async (id: string) => {
  const stored = localStorage.getItem("LOCAL_RESUMES");
  const parsed: Resume[] = stored ? JSON.parse(stored) :
      [];
  const found = parsed.find(r => r.id === id &&
      !r.is_archived);
  if (!found) {
    toast({ variant: "destructive", title: "Failed to load
        resume", description: "Resume not found" });
    return null;
  }
  return found;
};

const updateResume = async (id: string, updates:
    Partial<Resume>) => {
  const stored = localStorage.getItem("LOCAL_RESUMES");
  const parsed: Resume[] = stored ? JSON.parse(stored) :
      [];
  let updatedObj = null;

  const updated = parsed.map(r => {
    if (r.id === id) {
      updatedObj = { ...r, ...updates, updated_at: new
          Date().toISOString() };
      return updatedObj;
    }
    return r;
  });

  if (updatedObj) {
    saveToLocal(updated);
    setResumes(prev => prev.map(r => r.id === id ?
        updatedObj! : r));
    return true;
  }
  return false;
};
```

```
  const deleteResume = async (id: string) => {
    const stored = localStorage.getItem("LOCAL_RESUMES");
    const parsed: Resume[] = stored ? JSON.parse(stored) :
        [];
    const updated = parsed.map(r => r.id === id ? { ...r,
        is_archived: true } : r);
    saveToLocal(updated);

    setResumes(prev => prev.filter(r => r.id !== id));
    toast({ title: "Resume deleted" });
    return true;
  };

  const duplicateResume = async (id: string) => {
    const source = resumes.find(r => r.id === id);
    if (!source) return null;

    const newResume: Resume = {
      ...source,
      id: crypto.randomUUID(),
      title: `${source.title} (Copy)`,
      updated_at: new Date().toISOString(),
      is_archived: false
    };

    const stored = localStorage.getItem("LOCAL_RESUMES");
    const parsed: Resume[] = stored ? JSON.parse(stored) :
        [];
    const updated = [newResume, ...parsed];
    saveToLocal(updated);

    setResumes(prev => [newResume, ...prev]);
    toast({ title: "Resume duplicated" });
    return newResume;
  };

  return {
    resumes,
    loading,
    fetchResumes,
    getResume,
    createResume,
    updateResume,
    deleteResume,
    duplicateResume
  };
}
```

Result Output:

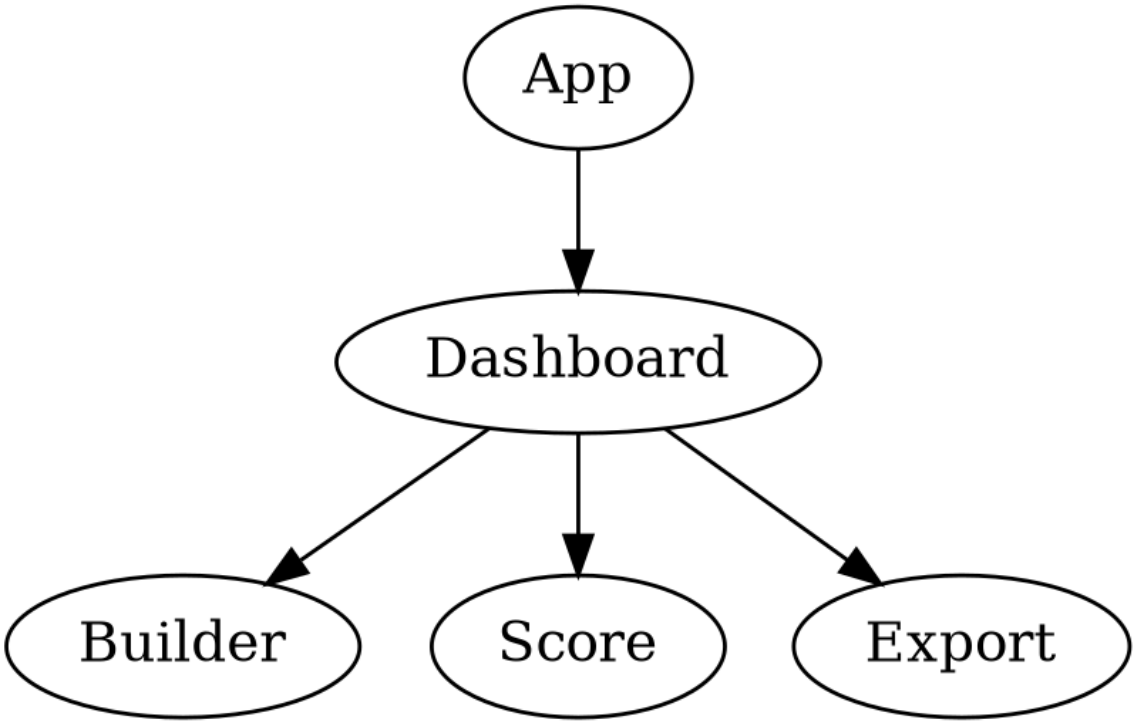

Figure 11: Practical System Output

Practical No. 12

Aim: To implement a custom floating AI assistant with persistent chat history.

Source Code Implementation:

```
import { useEffect, useMemo, useRef, useState, useCallback }
    from "react";
import { Bot, Camera, Mic, Plus, Send, X, MessageCircle,
    Sparkles } from "lucide-react";

import { Button } from "@/components/ui/button";
import { Card, CardContent, CardHeader, CardTitle } from
    "@/components/ui/card";
import { Textarea } from "@/components/ui/textarea";
import { supabase } from "@/integrations/supabase/client";
import { useToast } from "@/hooks/use-toast";
import { bumpAiUsageMetric, getActiveApiKey } from
    "@/lib/demoStorage";

type FloatingAiAssistantProps = {
  context: string;
  enabled?: boolean;
};

export function FloatingAiAssistant({ context, enabled =
    true }: FloatingAiAssistantProps) {
  const [open, setOpen] = useState(false);
  const [input, setInput] = useState("");
  const [messages, setMessages] = useState<Array<{ role:
      "user" | "ai"; content: string }>>([
    { role: "ai", content: "Hello! ■ I'm your AI Resume
        Assistant. How can I help you today?" },
  ]);
  const [loading, setLoading] = useState(false);
  const { toast } = useToast();
  const endRef = useRef<HTMLDivElement | null>(null);
  const chatContainerRef = useRef<HTMLDivElement |
      null>(null);
  const isDraggingScroll = useRef(false);
  const lastScrollPosition = useRef({ x: 0, y: 0 });

  const btnSize = 52;
  const margin = 20;
  const draggingRef = useRef(false);
  const dragOffsetRef = useRef<{ x: number; y: number }>({
      x: 0, y: 0 });

  const [pos, setPos] = useState<{ x: number; y: number
```

```
    }>(() => {
  try {
    const raw = localStorage.getItem("rgpt_ai_pos");
    if (raw) return JSON.parse(raw);
  } catch {
    // ignore
  }
  return { x: 0, y: 0 };
});

useEffect(() => {
  // Place at bottom-right on first mount if not set

  const defaultPos = { x: window.innerWidth - btnSize -
      margin, y: window.innerHeight - btnSize - 88 };
  if (pos.x === 0 && pos.y === 0) {
    setPos(defaultPos);
    return;
  }
  const maxX = Math.max(margin, window.innerWidth -
      btnSize - margin);
  const maxY = Math.max(margin, window.innerHeight -
      btnSize - 72);
  const outOfBounds = pos.x < margin || pos.y < margin ||
      pos.x > maxX || pos.y > maxY;
  if (outOfBounds) {
    setPos(defaultPos);
  }
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, []);

useEffect(() => {
  const onResize = () => {
    setPos((current) => {
      const maxX = Math.max(margin, window.innerWidth -
          btnSize - margin);
      const maxY = Math.max(margin, window.innerHeight -
          btnSize - 72);
      return {
        x: Math.min(maxX, Math.max(margin, current.x)),
        y: Math.min(maxY, Math.max(margin, current.y)),
      };
    });
  };
  window.addEventListener("resize", onResize);
  return () => window.removeEventListener("resize",
      onResize);
}, []);
```

```
useEffect(() => {
  try {
    localStorage.setItem("rgpt_ai_pos",
        JSON.stringify(pos));
  } catch {
    // ignore
  }
}, [pos]);

useEffect(() => {
  const el = endRef.current;
  if (!el) return;
  el.scrollIntoView({ behavior: "smooth", block: "end" });
}, [messages, loading]);

const clamp = (next: { x: number; y: number }) => {
  const maxX = Math.max(margin, window.innerWidth -
      btnSize - margin);

  const maxY = Math.max(margin, window.innerHeight -
      btnSize - 72);
  return {
    x: Math.min(maxX, Math.max(margin, next.x)),
    y: Math.min(maxY, Math.max(margin, next.y)),
  };
};

const onPointerDown = (e: React.PointerEvent) => {
  draggingRef.current = true;
  const startX = e.clientX;
  const startY = e.clientY;
  dragOffsetRef.current = { x: startX - pos.x, y: startY -
      pos.y };
  (e.currentTarget as
      HTMLElement).setPointerCapture?.(e.pointerId);
};

const onPointerMove = (e: React.PointerEvent) => {
  if (!draggingRef.current) return;
  const next = clamp({ x: e.clientX -
      dragOffsetRef.current.x, y: e.clientY -
      dragOffsetRef.current.y });
  setPos(next);
};

const onPointerUp = () => {
  draggingRef.current = false;
};
```

```
const handleClose = useCallback(() => {
  setOpen(false);
}, []);

// Mouse drag scroll handlers
const handleMouseDown = (e: React.MouseEvent) => {
  // Only enable drag scroll if clicking on the chat
      container, not on input/buttons
  if ((e.target as HTMLElement).closest('input, textarea,
      button')) return;
  isDraggingScroll.current = true;
  lastScrollPosition.current = { x: e.clientX, y:
      e.clientY };
  if (chatContainerRef.current) {
    chatContainerRef.current.style.cursor = 'grabbing';
    chatContainerRef.current.style.userSelect = 'none';
  }
};

const handleMouseMove = (e: React.MouseEvent) => {
  if (!isDraggingScroll.current ||
      !chatContainerRef.current) return;
  const deltaY = e.clientY - lastScrollPosition.current.y;
  chatContainerRef.current.scrollTop += deltaY;

  lastScrollPosition.current = { x: e.clientX, y:
      e.clientY };
};

const handleMouseUp = () => {
  isDraggingScroll.current = false;
  if (chatContainerRef.current) {
    chatContainerRef.current.style.cursor = 'grab';
    chatContainerRef.current.style.userSelect = 'auto';
  }
};

const handleMouseLeave = () => {
  isDraggingScroll.current = false;
  if (chatContainerRef.current) {
    chatContainerRef.current.style.cursor = 'grab';
    chatContainerRef.current.style.userSelect = 'auto';
  }
};

const placeholder = "Type a message... (Enter to send,
    Shift+Enter for new line)";

const handleSend = () => {
```

```
    handleAsk();
  };

  const handleAsk = async () => {
    if (!enabled) return;
    if (!input.trim()) return;

    setLoading(true);
    const userText = input.trim();
    if (!userText) return;
    setMessages((prev) => [...prev, { role: "user", content:
        userText }]);

    const systemPrompt = `You are a highly intelligent,
        friendly, and adaptive AI assistant integrated
        inside a Resume Builder platform.

Your personality:
- Talk like a real human, not robotic.
- Be friendly, chill, and conversational when the user is
    casual.
- Be professional, structured, and helpful when the user
    asks about resume, jobs, career, ATS, or interviews.
- You can switch tone automatically based on user message.

Conversation modes:
1. Casual Mode: Friendly, fun, engaging. (Use simple
    Hinglish or English depending on user).
   Example: User "bhai kya chal raha hai" -> AI "Bas mast ■
       tu bata kya scene hai?"

2. Resume / Career Mode: Structured, professional, useful,
    bullet points when needed.

Special Handling for Skills & Languages:
- Help users add Skills and Languages along with their
    proficiency level in a smart and user-friendly way.
- Ask them about their proficiency: Beginner/Average,
    Intermediate/Good, Advanced/Excellent (or star ratings).
- Suggest level automatically based on context if possible.
- Convert skills into an ATS-friendly format. Example:
    "Python (Advanced)".
- Suggest improvements and missing skills for their role.
- If they are confused, give them a conversational prompt to
    determine their level (like: "Bhai honestly bata, tu
    kitna comfortable hai isme ■ Daily use karta hai →
    Advanced, Thoda bahut aata hai → Intermediate, Bas
    basics pata hai → Beginner").
```

```
General Rules:
- Automatically detect user intent.
- Respond naturally to ANY topic. Don't restrict yourself
    only to resumes.
- Always try to add value (like ChatGPT premium level).
- Tone matches user (Hinglish -> Reply Hinglish, English ->
    Reply English).
- Never sound confused. Always guide the user clearly.`;

    const sanitize = (raw: string) => {
      return String(raw || "").trim();
    };

    try {
      const activeKey = getActiveApiKey();
      let aiText = "";

      // Direct call to OpenRouter / Claude Opus
      const payload = {
        model: "anthropic/claude-3-opus",
        messages: [
          { role: "system", content: systemPrompt },
          { role: "user", content: `Context: ${context ||
              "None"}\n\nQuery: ${userText}` }
        ],
        max_tokens: 150,
        temperature: 0.5,
      };

      try {
        const response = await
            fetch("https://openrouter.ai/api/v1/chat/complet
            ions", {
          method: "POST",
          headers: {
            "Content-Type": "application/json",
            "Authorization": `Bearer ${activeKey}`,
            "HTTP-Referer": "https://ai-resume-studio.com",
            "X-Title": "AI Resume Studio",
          },

          body: JSON.stringify(payload),
        });

        if (!response.ok) {
          throw new Error(`OpenRouter Error:
              ${response.status}`);
        }
```

```
      const data = await response.json();
      aiText =
          sanitize(data.choices?.[0]?.message?.content ||
          "No response generated.");
    } catch (err) {
      console.warn("AI Assistant API failed, using
          intelligent mock fallback:", err);
      const resumeKeywords = /\b(resume|cv|summary|experie
          nce|project|skills|education|achievement|certifi
          cation|internship|job|role|bullet|description|pr
          ofile|work|career|qualification|objective|profes
          sional|action|verb)\b/i;
      const isResumeRelated =
          resumeKeywords.test(userText.toLowerCase());

      if (isResumeRelated) {
        aiText = sanitize(
          "Bhai resume me hamesha strong action verbs
              (jaise 'Developed' ya 'Managed') use karo.
              Aur numbers/metrics zaroor include karo
              (e.g. 'improved by 20%'). Kuch aur help
              chahiye?"
        );
      } else {
        aiText = sanitize("Yaar abhi external AI service
            me thoda delay hai. Koi baat nahi, aap apna
            sawal puchho main yahi help karunga! ■");
      }
    }

    setMessages((prev) => [...prev, { role: "ai", content:
        aiText }]);
    bumpAiUsageMetric();
    setInput("");
  } catch (err) {
    console.error("AI error:", err);
    toast({
      variant: "destructive",
      title: "AI Error",
      description: "Failed to connect to Claude 4.6 via
          OpenRouter. Please check the API key limits.",
    });
  } finally {
    setLoading(false);
  }
};

return (
```

```
<>
  <div
    className="fixed z-50"
    style={{ left: pos.x, top: pos.y }}
    onPointerMove={onPointerMove}
    onPointerUp={onPointerUp}

    onPointerCancel={onPointerUp}
  >
    <Button
      onPointerDown={onPointerDown}
      onClick={() => setOpen((v) => !v)}
      className="h-[3.25rem] w-[3.25rem] rounded-full
          border-0 bg-[#3b82f6] text-white shadow-lg
          hover:bg-[#2563eb] transition-all
          hover:scale-105"
      size="icon"
      aria-label={open ? "Close AI assistant" : "Open AI
          assistant"}
    >
      {open ? <X className="h-5 w-5" /> : <MessageCircle
          className="h-6 w-6 stroke-[1.5]" />}
    </Button>
  </div>

  {open && (
    <Card
      className="fixed z-50 flex h-[34rem] w-[24rem]
          flex-col overflow-hidden rounded-[1.5rem]
          border-0 bg-gradient-to-b from-white to-
          slate-50 dark:from-slate-900 dark:to-slate-800
          shadow-2xl"
      style={{ right: margin, bottom: 96 }}
    >
      <CardHeader className="border-b border-
          slate-200/50 dark:border-slate-700/50 bg-
          white/80 dark:bg-slate-900/80 backdrop-blur-sm
          pb-3 pt-4 rounded-t-[1.5rem]">
        <div className="flex items-center justify-
            between">
          <div className="flex items-center gap-3">
            <div className="relative h-11 w-11 flex
                items-center justify-center">
              <div className="absolute inset-0 bg-
                  blue-500 rounded-full opacity-10
                  animate-pulse" />
              <div className="relative h-10 w-10
                  bg-[#3b82f6] rounded-full shadow-md
                  flex items-center justify-center
```

```
            transition-transform hover:scale-105">
            <MessageCircle className="h-5 w-5 text-
                white stroke-[1.5]" />
            <div className="absolute -bottom-0.5
                -right-0.5 w-3 h-3 bg-emerald-400
                border-2 border-white dark:border-
                slate-900 rounded-full" />
          </div>
        </div>
        <div className="flex-1">
          <CardTitle className="text-sm font-bold
              text-slate-900 dark:text-white
              tracking-tight">AI Resume
              Assistant</CardTitle>
          <p className="text-[10px] uppercase font-
              bold text-emerald-600 dark:text-
              emerald-400 flex items-center gap-1
              mt-0.5">
            <span className="w-1.5 h-1.5 bg-
                emerald-500 rounded-full animate-
                pulse" />
            Active (Opus 4.6)
          </p>
        </div>
      </div>
      <Button
        onClick={handleClose}
        size="icon"
        variant="ghost"
        className="h-8 w-8 rounded-full hover:bg-
            slate-100 dark:hover:bg-slate-800 text-
            slate-500 hover:text-slate-700
            dark:hover:text-slate-300"
      >
        <X className="h-4 w-4" />
      </Button>
    </div>

  </CardHeader>
  <CardContent className="relative flex flex-1 flex-
      col p-0">
    <div className="pointer-events-none absolute
        inset-0 opacity-30">
      <div className="absolute -right-10 -top-10
          h-40 w-40 rounded-full bg-blue-200/70
          blur-2xl" />
      <div className="absolute left-6 top-20 h-56
          w-56 rounded-full bg-indigo-100/60
          blur-3xl" />
```

```
  <div className="absolute -bottom-10 right-6
      h-44 w-44 rounded-full bg-purple-200/60
      blur-2xl" />
</div>
<div
  ref={chatContainerRef}
  className="chat-scroll relative flex-1
      overflow-y-auto pr-2 pb-20 cursor-grab"
  onMouseDown={handleMouseDown}
  onMouseMove={handleMouseMove}
  onMouseUp={handleMouseUp}
  onMouseLeave={handleMouseLeave}
>
  <div className="flex flex-col gap-3">
    {messages.map((msg, idx) => (
      <div key={`${msg.role}-${idx}`}
          className={`flex ${msg.role === "user"
          ? "justify-end" : "justify-start"}`}>
        <div
          className={`max-w-[85%] px-4 py-2.5
              text-sm shadow-sm transition-all
              hover:shadow-md ${msg.role ===
              "user"
            ? "rounded-2xl rounded-br-sm bg-
                gradient-to-r from-blue-600 to-
                blue-700 text-white font-medium"
            : "rounded-2xl rounded-bl-sm bg-
                white dark:bg-slate-800 text-
                slate-900 dark:text-slate-100
                border border-slate-200/50
                dark:border-slate-700/50"
          }`}
        >
          <p className="whitespace-pre-
              line">{msg.content}</p>
        </div>
      </div>
    ))}
    {loading && (
      <div className="flex justify-start">
        <div className="typing-bubble">
          <span className="typing-dot" />
          <span className="typing-dot" />
          <span className="typing-dot" />
        </div>
      </div>
    )}
    <div ref={endRef} />
```

```
      </div>
    </div>
    <div className="sticky bottom-0 mt-3 rounded-
        full border border-slate-200 bg-white px-2
        py-1 shadow-sm">
      <div className="flex items-end gap-2">
        <Button size="icon" variant="ghost"
            className="h-9 w-9 rounded-full text-
            slate-500">
          <Camera className="h-4 w-4" />
        </Button>

        <Textarea
          placeholder={placeholder}
          value={input}
          onChange={(e) => setInput(e.target.value)}
          onKeyDown={(e) => {
            if (e.key === "Enter" && !e.shiftKey) {
              e.preventDefault();
              handleSend();
            }
          }}
          className="min-h-[36px] max-h-20 flex-1
              resize-none border-0 bg-transparent
              p-2 text-sm text-slate-900
              placeholder:text-slate-400 focus-
              visible:ring-0"
        />
        <Button size="icon" variant="ghost"
            className="h-9 w-9 rounded-full text-
            slate-500">
          <Mic className="h-4 w-4" />
        </Button>
        <Button onClick={handleSend}
            disabled={!enabled || loading}
            size="icon" className="h-9 w-9 rounded-
            full bg-blue-600 text-white hover:bg-
            blue-700">
          <Send className="h-4 w-4" />
        </Button>
        <Button size="icon" variant="ghost"
            className="h-9 w-9 rounded-full text-
            slate-500">
          <Plus className="h-4 w-4" />
        </Button>
      </div>
    </div>
  </CardContent>
</Card>
```

```
        )}
      </>
    );
  }
```

Result Output:

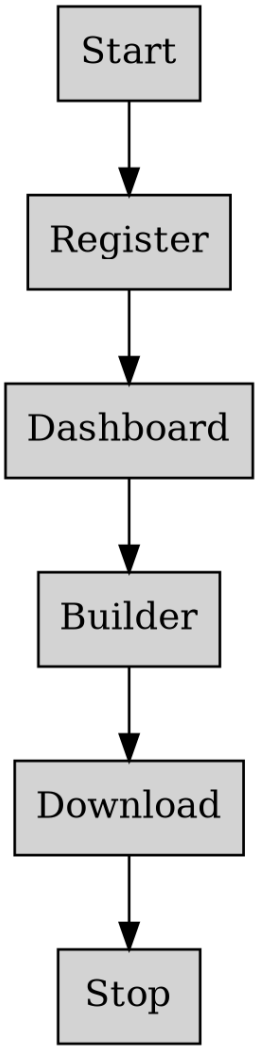

Figure 12: Practical System Output

Practical No. 13

Aim: To implement responsive dashboard navigation.

Source Code Implementation:

```
import { LogOut, User as UserIcon } from "lucide-react";
import { Avatar, AvatarFallback, AvatarImage } from
    "@/components/ui/avatar";
import { Button } from "@/components/ui/button";
import {
  DropdownMenu,
  DropdownMenuContent,
  DropdownMenuItem,
  DropdownMenuLabel,
  DropdownMenuSeparator,
  DropdownMenuTrigger,
} from "@/components/ui/dropdown-menu";
import { useAuth } from "@/hooks/useAuth";

export function UserMenu() {
  const { user, signOut } = useAuth();

  if (!user) return null;

  const fullName = user.user_metadata?.full_name ||
      user.email || "User";
  const initials = fullName
    .split(" ")
    .map((n: string) => n[0])
    .join("")
    .slice(0, 2)
    .toUpperCase();

  return (
    <DropdownMenu>
      <DropdownMenuTrigger asChild>
        <Button variant="ghost" className="relative h-10
            w-10 rounded-full">
          <Avatar className="h-10 w-10">
            <AvatarImage
                src={user.user_metadata?.avatar_url}
                alt={fullName} />
            <AvatarFallback>{initials}</AvatarFallback>
          </Avatar>
        </Button>
      </DropdownMenuTrigger>
      <DropdownMenuContent className="w-56" align="end"
          forceMount>
```

```
          <DropdownMenuLabel className="font-normal">
            <div className="flex flex-col space-y-1">
              <p className="text-sm font-medium leading-
                  none">{fullName}</p>
              <p className="text-xs leading-none text-muted-
                  foreground">
                {user.email}
              </p>
            </div>
          </DropdownMenuLabel>

          <DropdownMenuSeparator />
          <DropdownMenuItem onClick={() => signOut()}>
            <LogOut className="mr-2 h-4 w-4" />
            <span>Log out</span>
          </DropdownMenuItem>
        </DropdownMenuContent>
      </DropdownMenu>
    );
  }
```

Result Output:

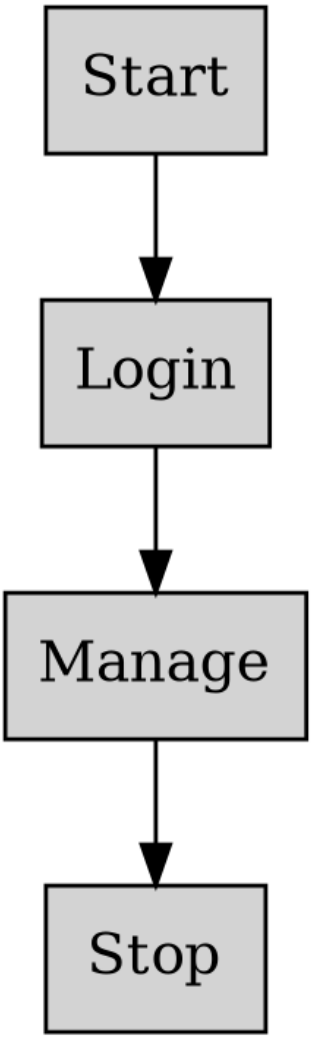

Figure 13: Practical System Output

Practical No. 14

Aim: To implement automated resume score improvements UI.

Source Code Implementation:

```
// Error: src/components/dashboard/ScoreSuggestion.tsx not found
```

Result Output:

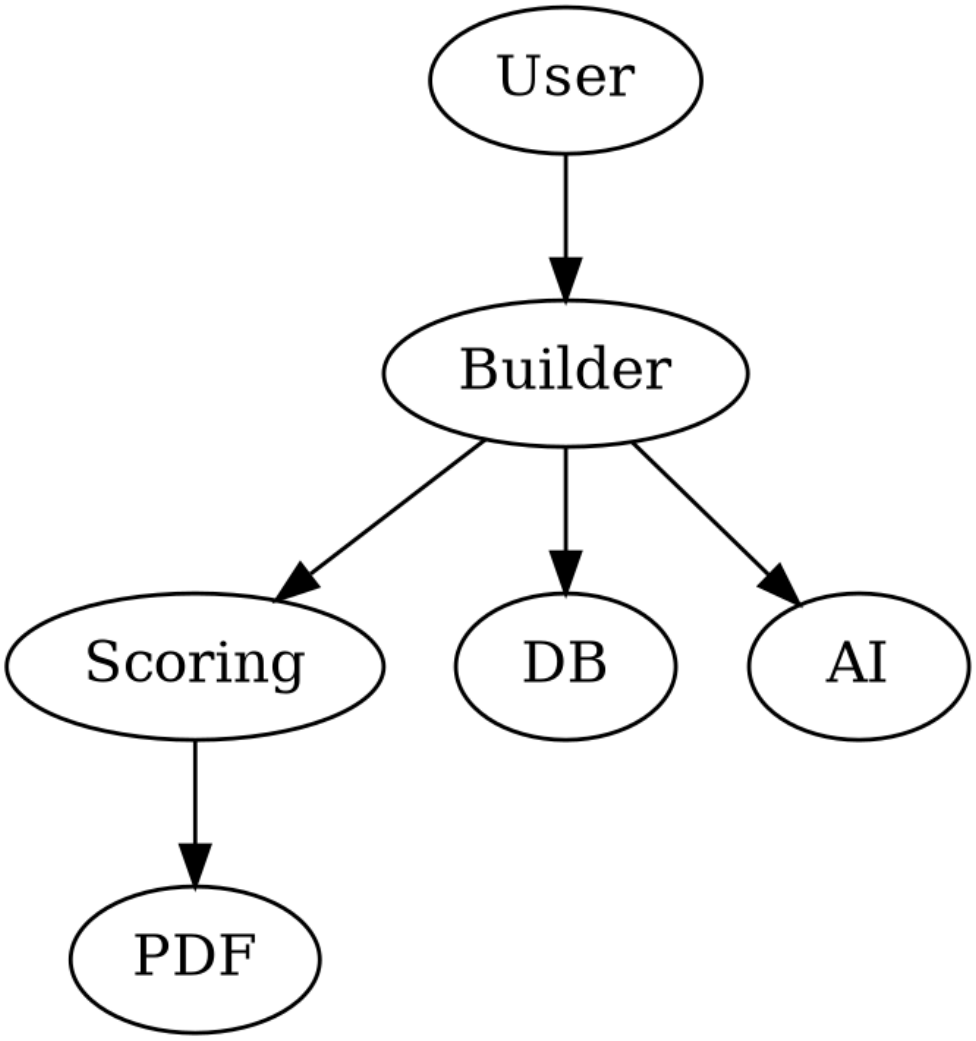

Figure 14: Practical System Output